School of Computing and Augmented Intelligence
ASU IRA A. FULTON SCHOOLS OF engineering

CSE 446/598
Software Integration and Engineering

Unit 1
Service Standards and Service Development

Lecture 1-5
RESTful Service Development and Case Studies

Dr. Yinong Chen

Lecture Roadmap

Previous lecture:
Using WCF template

- **Developing REST Services Without WCF Template**
  - Develop RESTful Web API (CSE445 Review)
  - Develop a gRPC Service and Client
  - Develop a RESTful Service in Python
- **Case Study: Image Verifier in RESTful Service**
  - RESTful Service of a Random String Generator
  - RESTful Service of an Image Verifier
  - TryIt page calling RESTful Services
- **RESTful services and Applications in ASP .Net Core**

Creating RESTful Web APIs without WCF

https://www.tutorialsteacher.com/webapi/create-web-api-project

Step 1: Choose right template:

web application

All languages | All platforms | All project types

ASP.NET Core Web Application

Project templates for creating ASP.NET Core web apps and web APIs for Windows, Linux and macOS using .NET Core or .NET Framework. Create web apps with Razor Pages, MVC, or Single Page Apps (SPA) using Angular, React, or React + Redux.

C# Linux macOS Windows Cloud Service Web

ASP.NET Web Application (.NET Framework)

Project templates for creating ASP.NET applications. You can create ASP.NET Web Forms, MVC, or Web API applications and add many other features in ASP.NET.

C# Windows Cloud Web

Step 2:

Project name

MyWebApi

Location

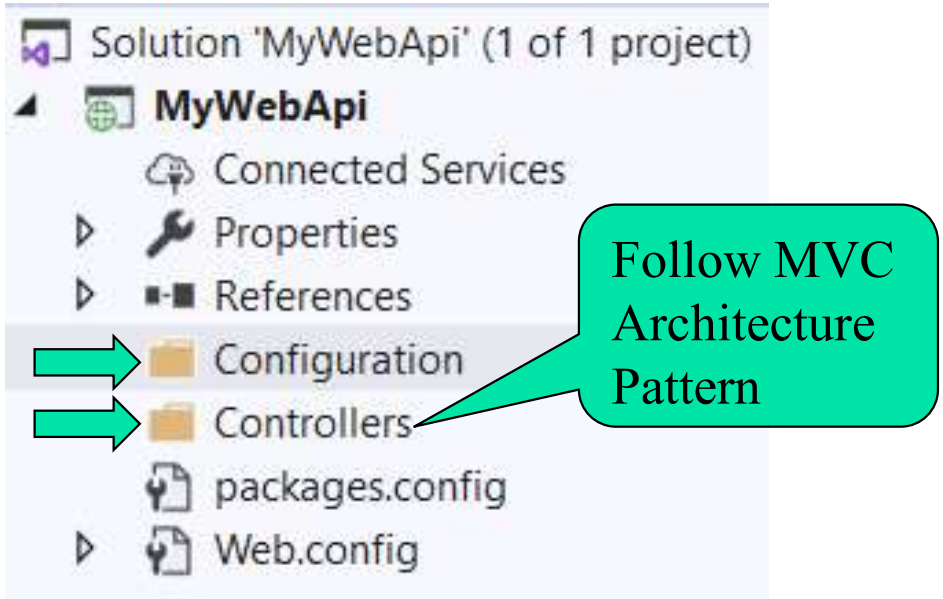
Step 3:

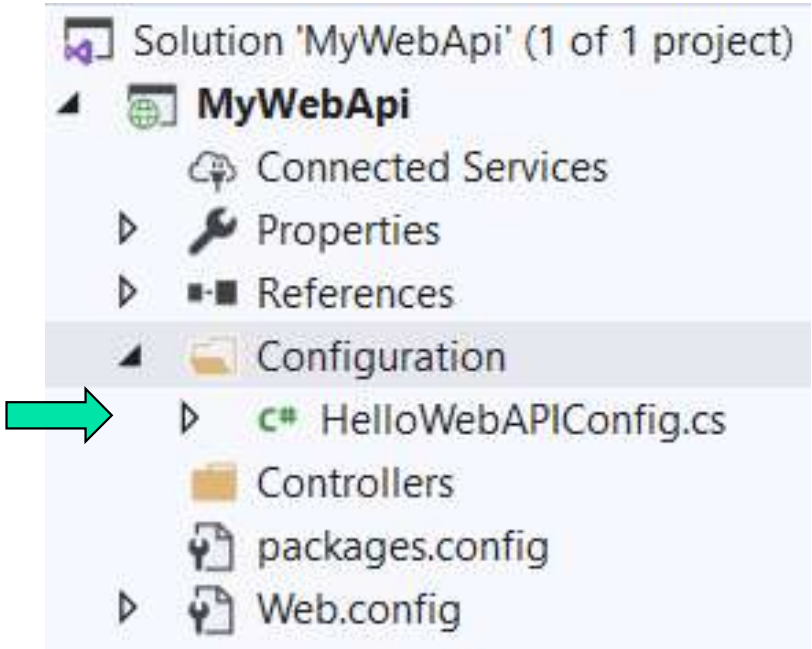
Empty

An empty project template for creating ASP.NET applications. This template does not have any content in it.

Add Controllers and Configuration Folders

- Right click project name and add two New Folders into the project
- Right click Configuration folder and add a class HelloWebAPIConfig





- Configuration folder holds a class for registering and configuring the service
- Controller folder holds a class that implement the service

Copy the code into HelloWebAPIConfig class

```
using System;
using System.Web.Http;
namespace HelloWebAPI.Configuration
{
    public static class HelloWebAPIConfig
    {
        public static void Register(HttpConfiguration config)
        {
            config.MapHttpAttributeRoutes(); // Web API routes
            config.Routes.MapHttpRoute(
                name: "DefaultApi",
                routeTemplate: "api/{controller}/{id}",
                defaults: new { id = RouteParameter.Optional }
            );
        }
    }
}
```

Register and define the path to the code

Copy this code into Global.asax file

```
using System;
using System.Web.Http;
using HelloWebAPI.Configuration;

namespace MyWebApi
{
    public class Global : System.Web.HttpApplication
    {
        protected void Application_Start(object sender, EventArgs e)
        {
            GlobalConfiguration.Configure(HelloWebAPIConfig.Register);
        }
    }
}
```

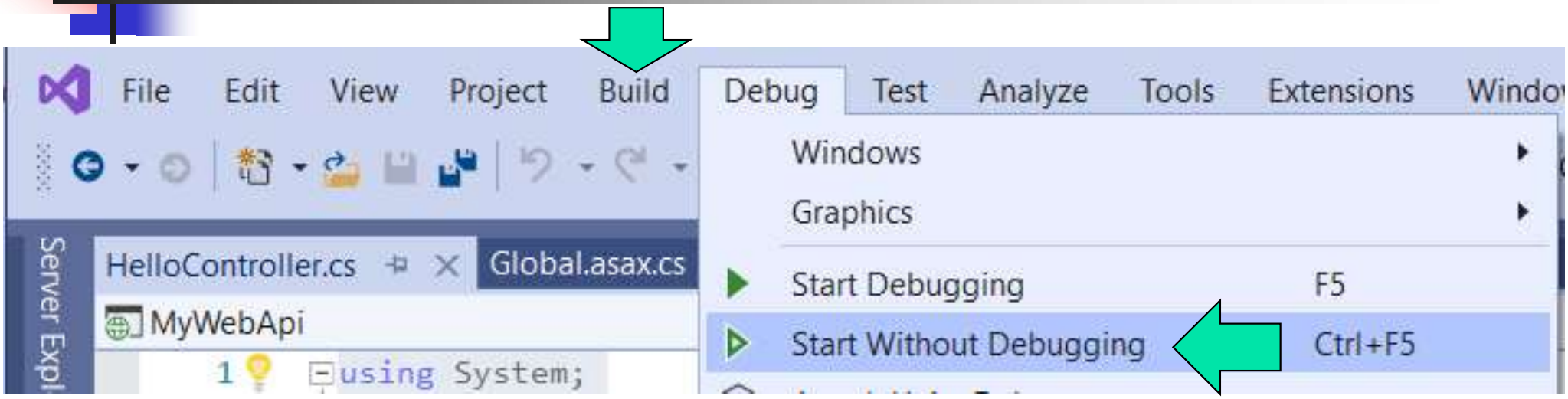
Application_Start event handler: When the application starts, it runs the code to register the api.

Copy the code into HelloController.cs

```
using System;
using System.Web.Http;

namespace MyWebApi.Controllers
{
    public class HelloController : ApiController
    {
        public string Get()
        {
            return "Hello World";
        }
    }
}
```

You can add any service code here. For example, temperature conversion service.

Build and then Start



- The browser will start at the address: https://localhost:44379/
  → HTTP Error 403.14 - Forbidden
- You type the full address into your web browser: https://localhost:44379/api/hello

Deployed to venus server:

http://venus.sod.asu.edu/WSRepository/Webapis/HelloWebApi/api/hello

Lecture Roadmap

Previous lecture:
Using WCF template

- **Developing REST Services Without WCF template**
  - Develop RESTful Web API (CSE445 Review)
  - Develop a gRPC Service and Client
  - Develop a RESTful Service in Python
- **Case Study: Image Verifier in RESTful Service**
  - RESTful Service of a Random String Generator
  - RESTful Service of an Image Verifier
  - TryIt page calling RESTful Services
- **RESTful services and Applications in ASP .Net Core**

Creating Google gRPC Service and Client in Visual Studio Core

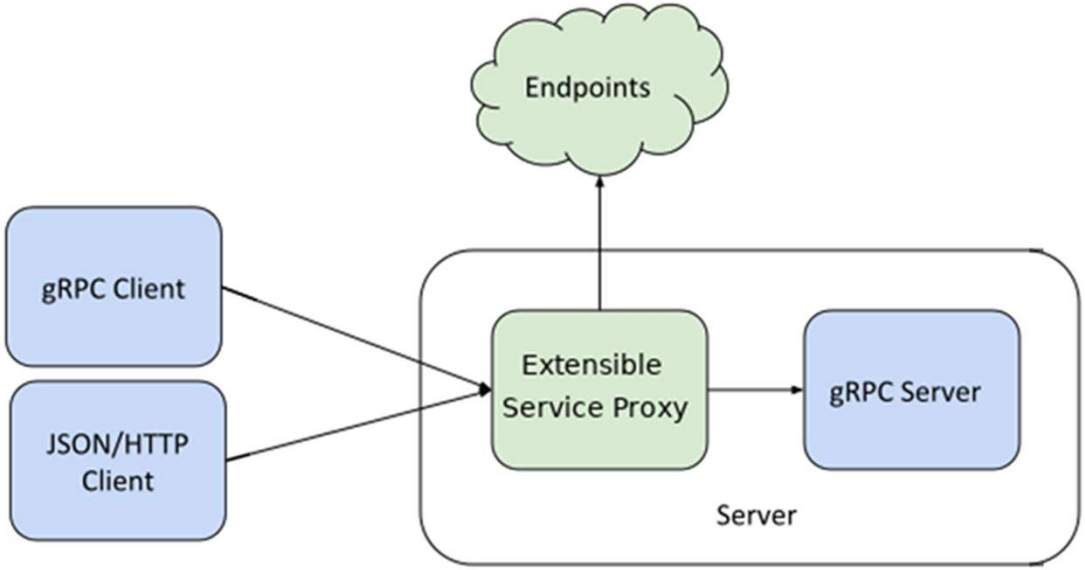
Please read the slides together with this tutorial:
https://docs.microsoft.com/en-us/aspnet/core/tutorials/grpc/grpc-start?view=aspnetcore-5.0&tabs=visual-studio

- Create a gRPC Service.
- Create a gRPC client.
- Test the gRPC client service with the gRPC Greeter service.

Cloud Endpoints for gRPC

1) https://cloud.google.com/endpoints/docs/grpc/about-grpc
2) https://www.grpc.io/docs/

- gRPC is a high performance, open-source universal RPC framework, developed by Google.
- In gRPC, a client application can directly call methods on a server (on a different machine) as if it was a local object, making it easier to create distributed applications and services.
- gRPC exposes endpoints to outside to enable different types of clients.
  - gRPC client
  - HTTP / JSON clients



- Example:
  using Visual Studio to develop a gRPC service and client

A gRPC service supportef languages

Supported programming languages for developing gRPC services include:

C++

C#/.NET

Dart

Go

Java

Kotlin

Node.js (JavaScript)

Objective-C

PHP

Python

Ruby

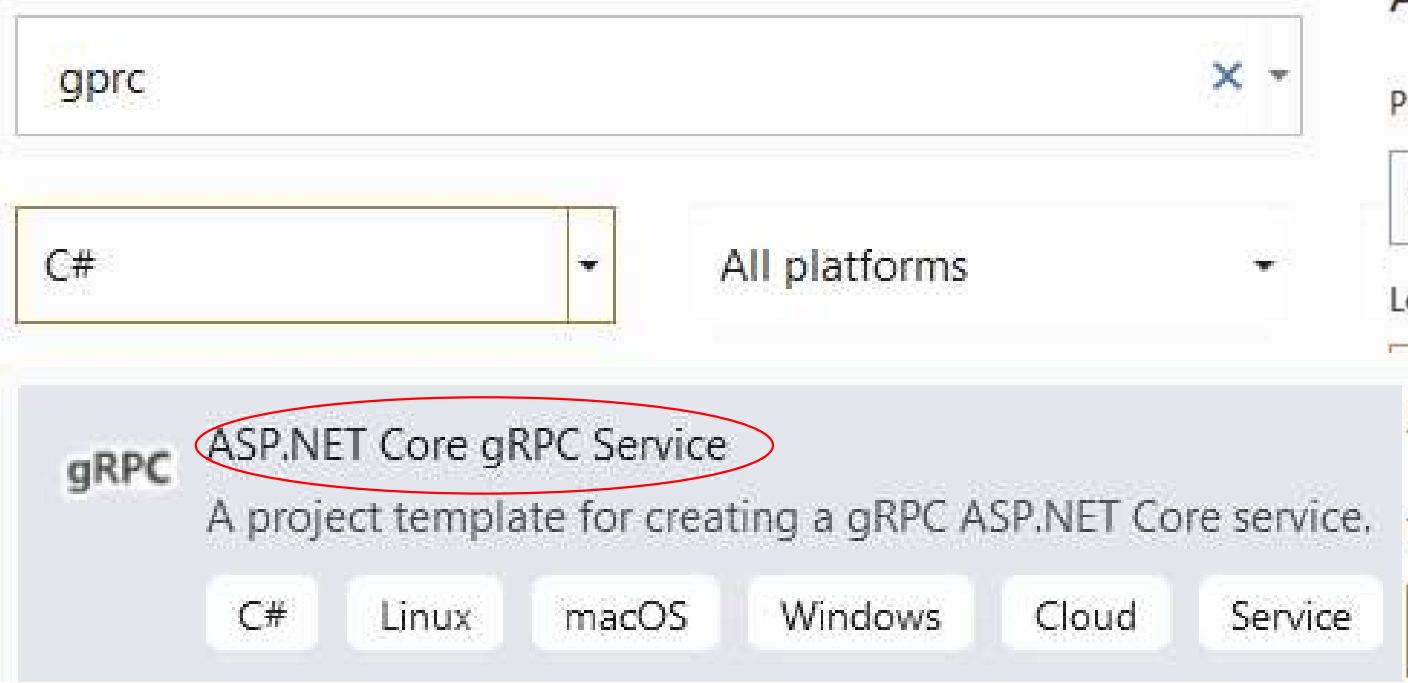
Swift

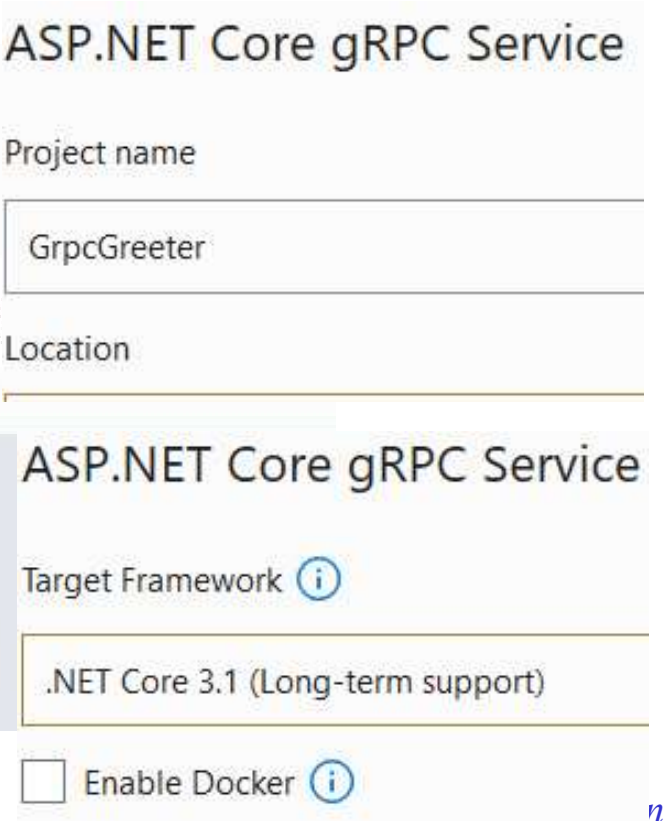
…

Creating gRPC Service in Visual Studio

https://docs.microsoft.com/en-us/aspnet/core/tutorials/grpc/grpc-start?view=aspnetcore-5.0&tabs=visual-studio

- Start Visual Studio and select **Create a new project**.
- In the **Create a new project** dialog, select **gRPC Service** and select **Next**

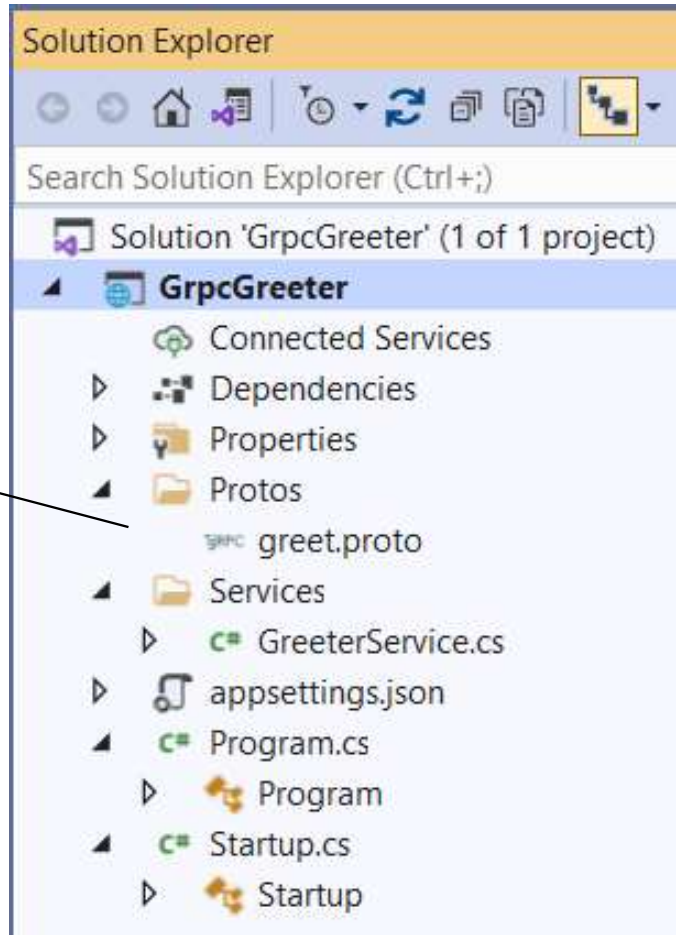




gRPC Service Interface Definition

greet.proto

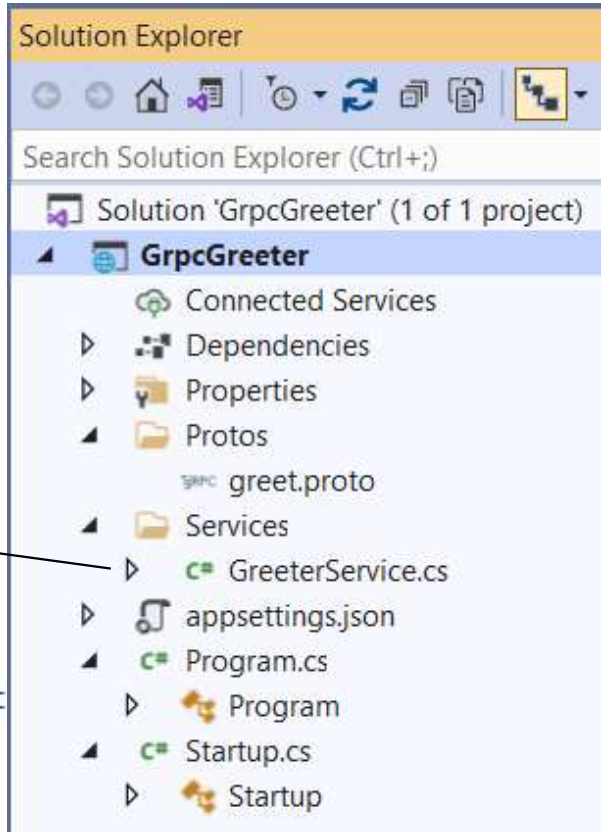
```
syntax = "proto3";

option csharp_namespace = "GrpcGreeter";

package greet;

// The greeting service definition.
service Greeter {
  // Sends a greeting
  rpc SayHello (HelloRequest) returns (HelloReply);
}

// The request message containing the user's name.
message HelloRequest {
  string name = 1;
}

// The response message containing the greetings.
message HelloReply {
  string message = 1;
}
```



Define Interface using Protocol Buffer (CSE445)

gRPC Service Implementation Code

```
namespace GrpcGreeter
{
    4 references
    public class GreeterService : Greeter.GreeterBase
    {
        private readonly ILogger<GreeterService> _logger;
        0 references
        public GreeterService(ILogger<GreeterService> logger)
        {
            _logger = logger;
        }

        3 references
        public override Task<HelloReply> SayHello(HelloRequest request
        {
            return Task.FromResult(new HelloReply
            {
                Message = "Hello " + request.Name
            });
        }
    }
}
```



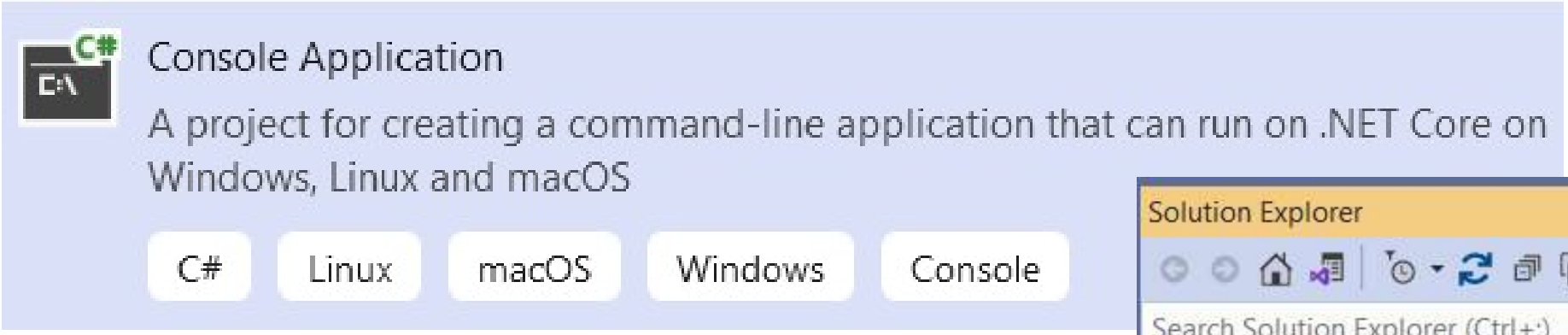
Write service code in C#

Start the gRPC service at https://localhost:5001

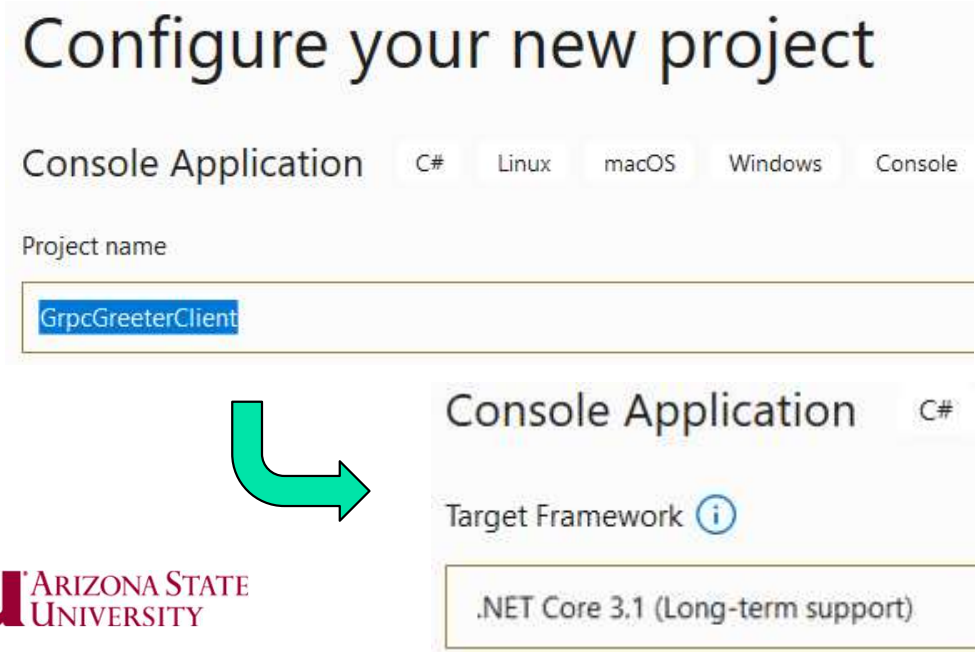
- Press Ctrl+F5 or Start Without Debugging
- Visual Studio displays a dialog for configuring to use SSL.
  - Click Yes.
- A Security Waring dialog for installing security certificate
  - Click Yes.
- Then, Visual Studio will run the service in IIS Express. The information will be displayed in Console window, showing
  - Now listening on: https://localhost:5001

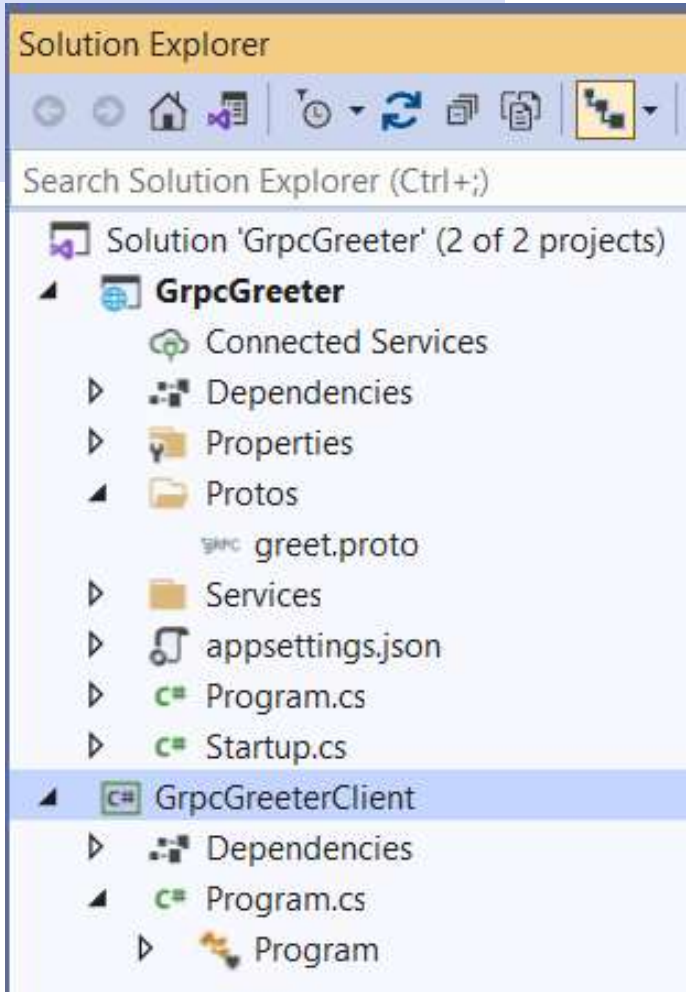
Create Client Project

- Right click Solution and choose to add a New Project



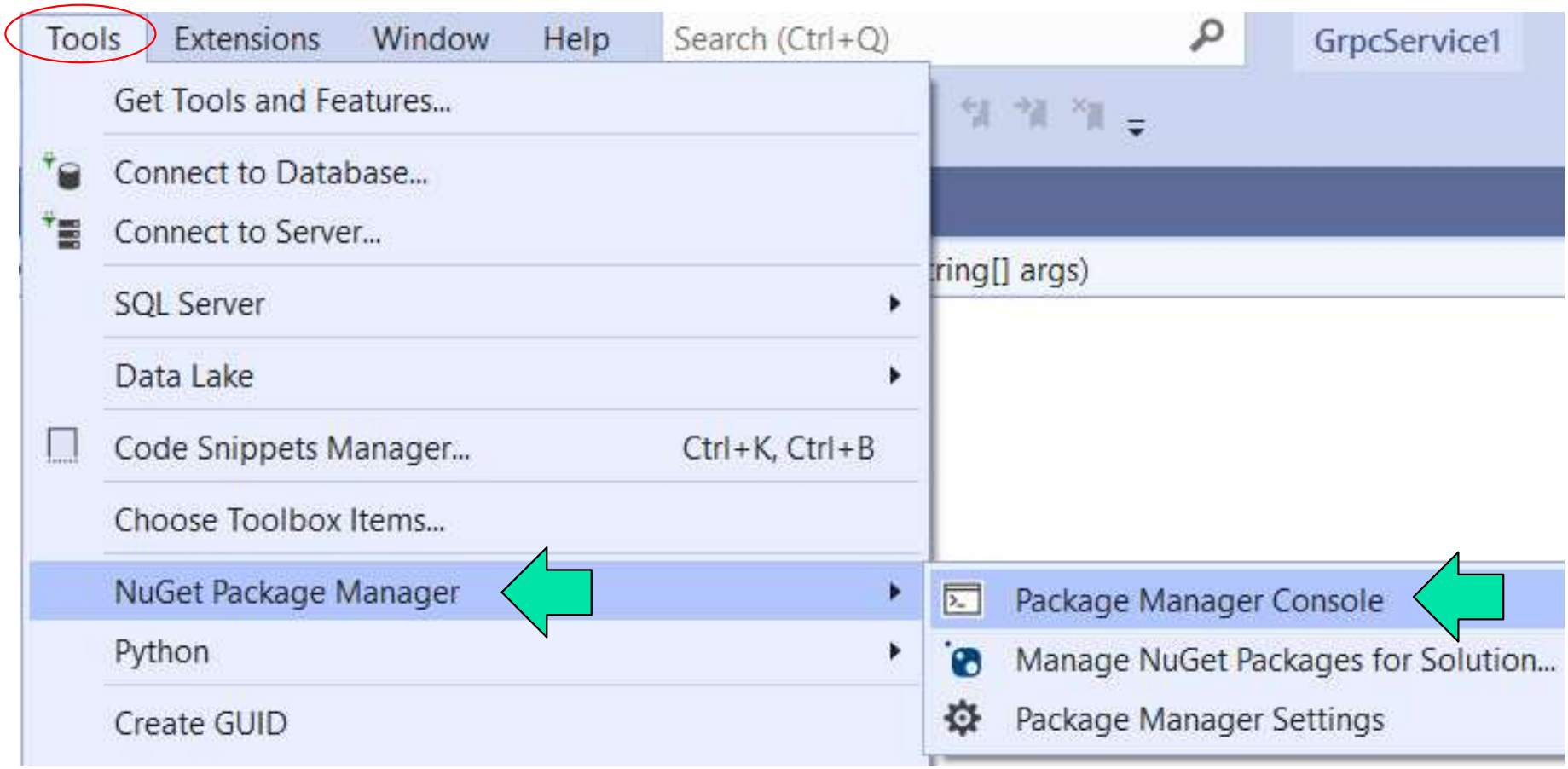
- Name the Project: GrpcGreeterClient





Add Required Packages to Client Project

- In Visual Studio Menu, Select:
- Tools → NuGet Package Manager → Package Manager Console



Install Three Packages by Commands:

At the lower window in Visual Studio:

Install-Package Grpc.Net.Client
Install-Package Google.Protobuf
Install-Package Grpc.Tools

Package Manager Console

Package source: All Default project: GrpcGreeter

```
Each package is licensed to you by its owner. NuGet is not responsible for,
packages may include dependencies which are governed by additional licenses.

Package Manager Console Host Version 5.10.0.7240

Type 'get-help NuGet' to see all available NuGet commands.

PM> Install-Package Grpc.Net.Client
```

Package Manager Console

Package source: All Default project: GrpcGreeter

```
Successfully uninstalled 'System.Diagnostics.DiagnosticSource 4.5.1' from
Successfully installed 'Grpc.Core.Api 2.37.0' to GrpcGreeter
Successfully installed 'Grpc.Net.Client 2.38.0' to GrpcGreeter
Successfully installed 'Grpc.Net.Common 2.38.0' to GrpcGreeter
Executing nuget actions took 350 ms
Time Elapsed: 00:00:01.7429817
PM> Install-Package Google.Protobuf
```

Package Manager Console

Package source: All Default project: GrpcGreeter

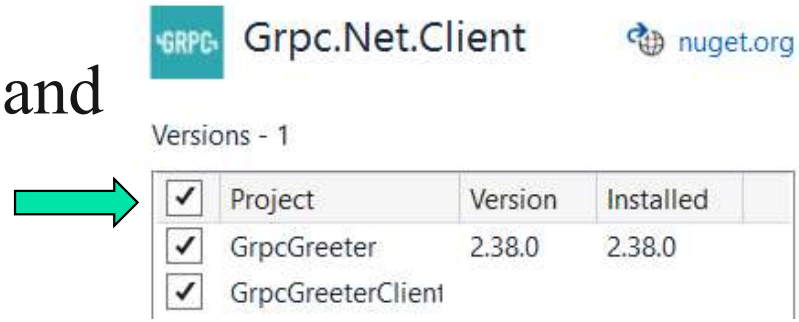
```
Successfully uninstalled 'Google.Protobuf 3.14.0' from GrpcGreeter
Successfully uninstalled 'System.Runtime.CompilerServices.Unsafe 4.5.2' from
Successfully installed 'Google.Protobuf 3.17.3' to GrpcGreeter
Executing nuget actions took 136 ms
Time Elapsed: 00:00:00.5130544
PM> Install-Package Grpc.Tools
```

Another Way of Installing NuGet Packages

Manage NuGet Packages option to install packages

- Right-click the project in **Solution Explorer** > **Manage NuGet Packages**
- Select Browse tab and enter Grpc.Net.Client in the search box

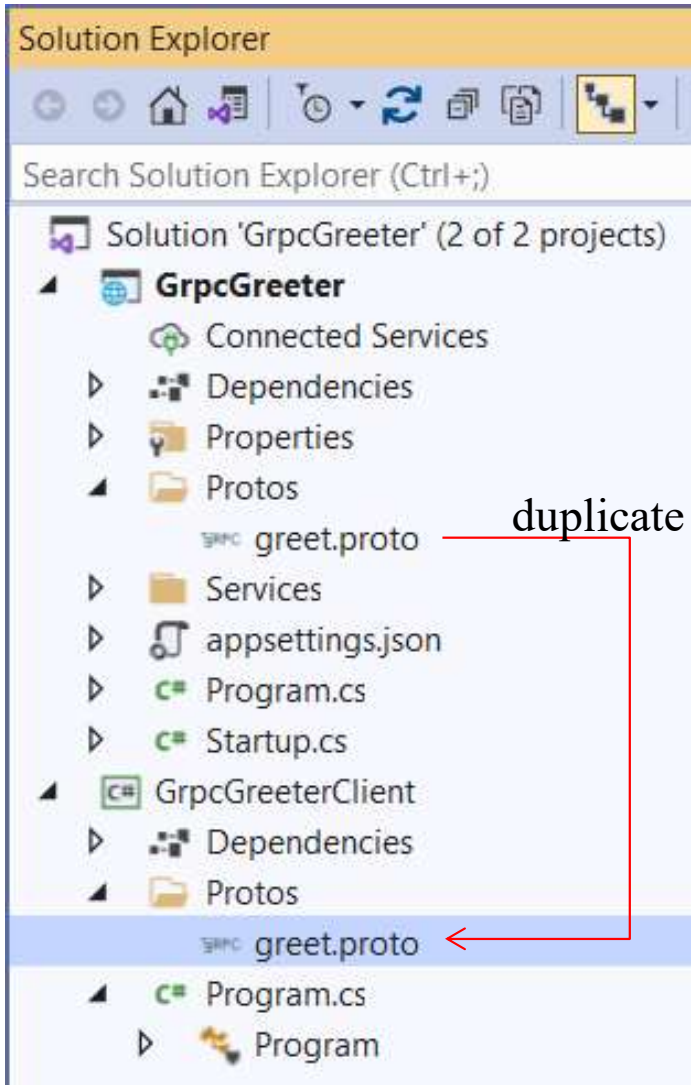
- Select the **Grpc.Net.Client** package and select **Install.**
- Repeat for **Google.Protobuf** and **Grpc.Tools**.



Create Client Project (Contd.)

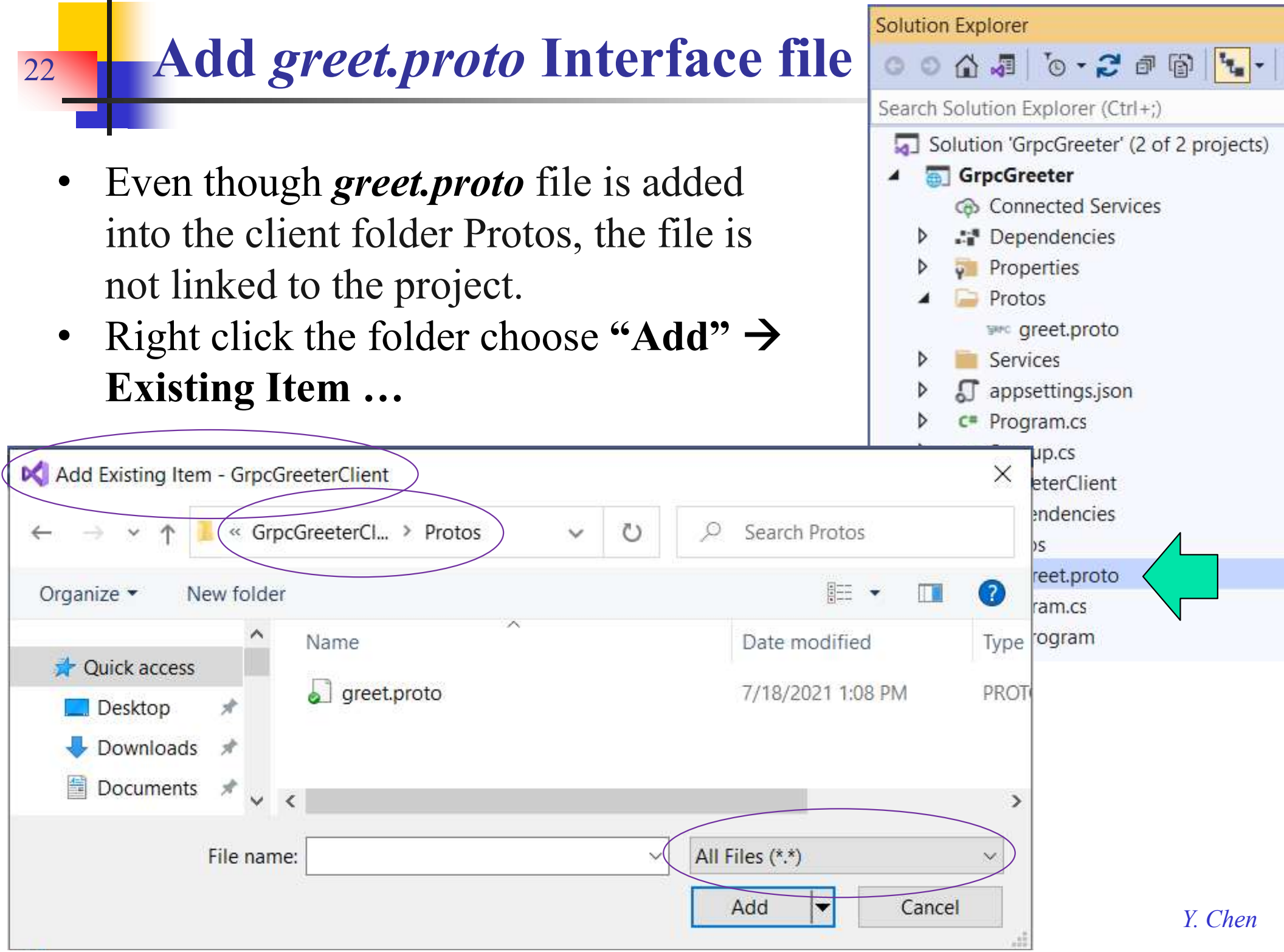
- Create a *Protos* folder in the gRPC client project
- Copy the *Protos\greet.proto* interface file from the gRPC Greeter service to the gRPC client project.
- In the Client project, update the namespace inside the ***greet.proto*** file to the project's namespace:

option csharp_namespace = "GrpcGreeterClient";



Add *greet.proto* Interface file

- Even though ***greet.proto*** file is added into the client folder Protos, the file is not linked to the project.
- Right click the folder choose **"Add" → Existing Item …**



Edit the *GrpcGreeterClient.csproj* project file

- Right-click Client project and select Edit Project File.
- Add / or change the new <ItemGroup> element:

```
<ItemGroup>
  <Protobuf Include="Protos\greet.proto" GrpcServices="Client" />
</ItemGroup>
```

```
<ItemGroup>
  <PackageReference Include="Google.Protobuf" Version="3.17.3" />
  <PackageReference Include="Grpc.Net.Client" Version="2.38.0" />
  <PackageReference Include="Grpc.Tools" Version="2.38.1">
    <PrivateAssets>all</PrivateAssets>
    <IncludeAssets>runtime; build; native; contentfiles; analyzers;
  </PackageReference>
</ItemGroup>
<ItemGroup>
  <None Update="Protos\greet.proto">
    <GrpcServices>Server</GrpcServices>
  </None>
</ItemGroup>
<ItemGroup>
  <Protobuf Include="Protos\greet.proto" GrpcServices="Client" />
</ItemGroup>
```

Write the Greeter client code

Asynchronous Service Call

```
using System; using System.Net.Http;
using System.Threading.Tasks; using Grpc.Net.Client;
namespace GrpcGreeterClient {
  class Program  {
    static async Task Main(string[] args)  {
      // The port number(5001) must match the port of the gRPC server.
      using var channel = GrpcChannel.ForAddress("https://localhost:5001");
      var client =  new Greeter.GreeterClient(channel);
      var reply = await client.SayHelloAsync(new HelloRequest {Name =
                                                          "GreeterClient"});
      Console.WriteLine("Greeting: " + reply.Message);
      Console.WriteLine("Press any key to exit...");
      Console.ReadKey();
    }
  }
}
```

Test Service and Client on Localhost

- Right click service project and choose "Set as Startup Project"
  - Start Without Debugging

```
C:\Users\ychen10\Dropbox (ASU)\Current in Dropbox\Teach All\CSE445-446 Programs
info: Microsoft.Hosting.Lifetime[0]
      Now listening on: https://localhost:5001
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: C:\Users\CSE445-446 Programs\Core services
```

- Right click client project and choose "Set as Startup Project"
  - Start Without Debugging

```
C:\windows\system32\cmd.exe
Greeting: Hello GreeterClient
Press any key to exit...
```

Lecture Roadmap

Previous lecture:
Using WCF template

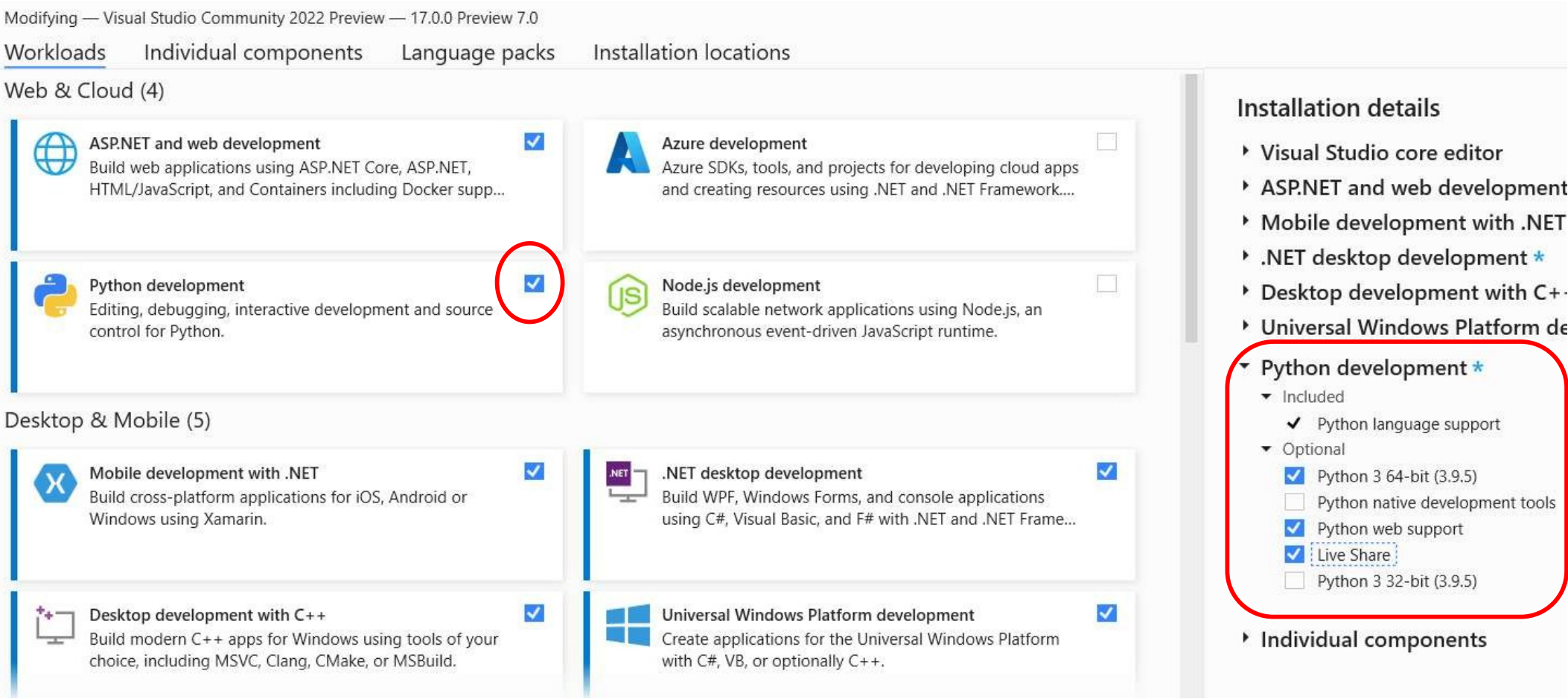
- **Developing REST Services Without WCF template**
  - Develop RESTful Web API (CSE445)
  - Develop a gRPC Service and Client
  - Develop a RESTful Service in Python
- **Case Study: Image Verifier in RESTful Service**
  - RESTful Service of a Random String Generator
  - RESTful Service of an Image Verifier
  - TryIt page calling RESTful Services
- **RESTful services and Applications in ASP .Net Core**

Service Development in Python

- Visual Studio Python Module
- PyCharm: Intelligent Python IDE with refactorings, debugger, code completion, on-the-fly code analysis, and coding productivity orientation (https://www.jetbrains.com/pycharm/download/).
- VSCode (Mac)
- PyDev for Eclipse
  - https://www.pydev.org/manual_101_install.html
  - If your comfortable with Eclipse for Java then there is very little learning curve

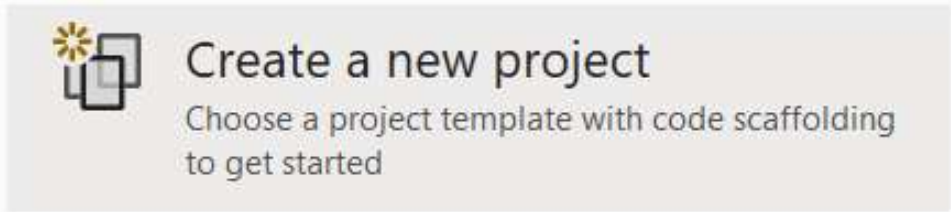
Creating Python Web Service Using Flask

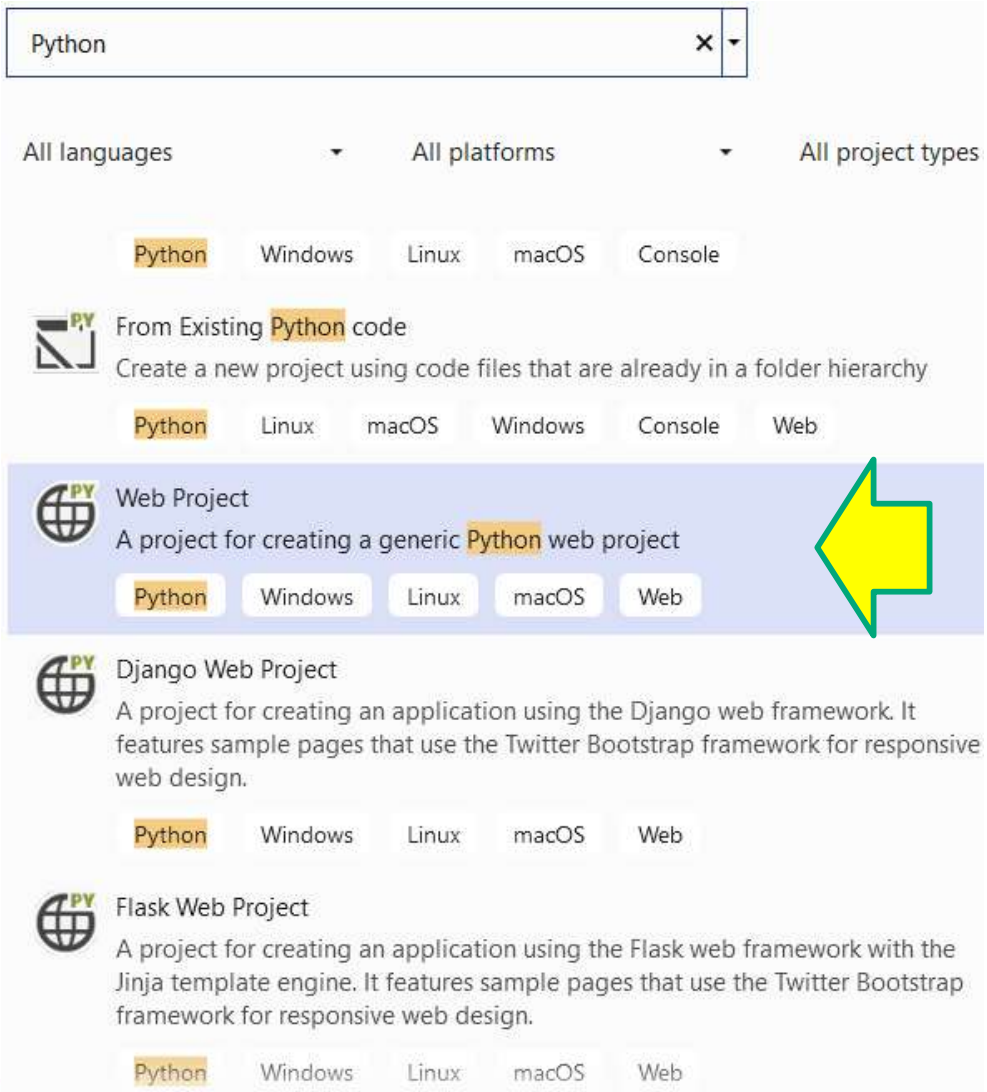
- Tutorial
  https://docs.microsoft.com/en-us/visualstudio/ide/quickstart-python?view=vs-2022
- Run Visual Studio Installer to install Python

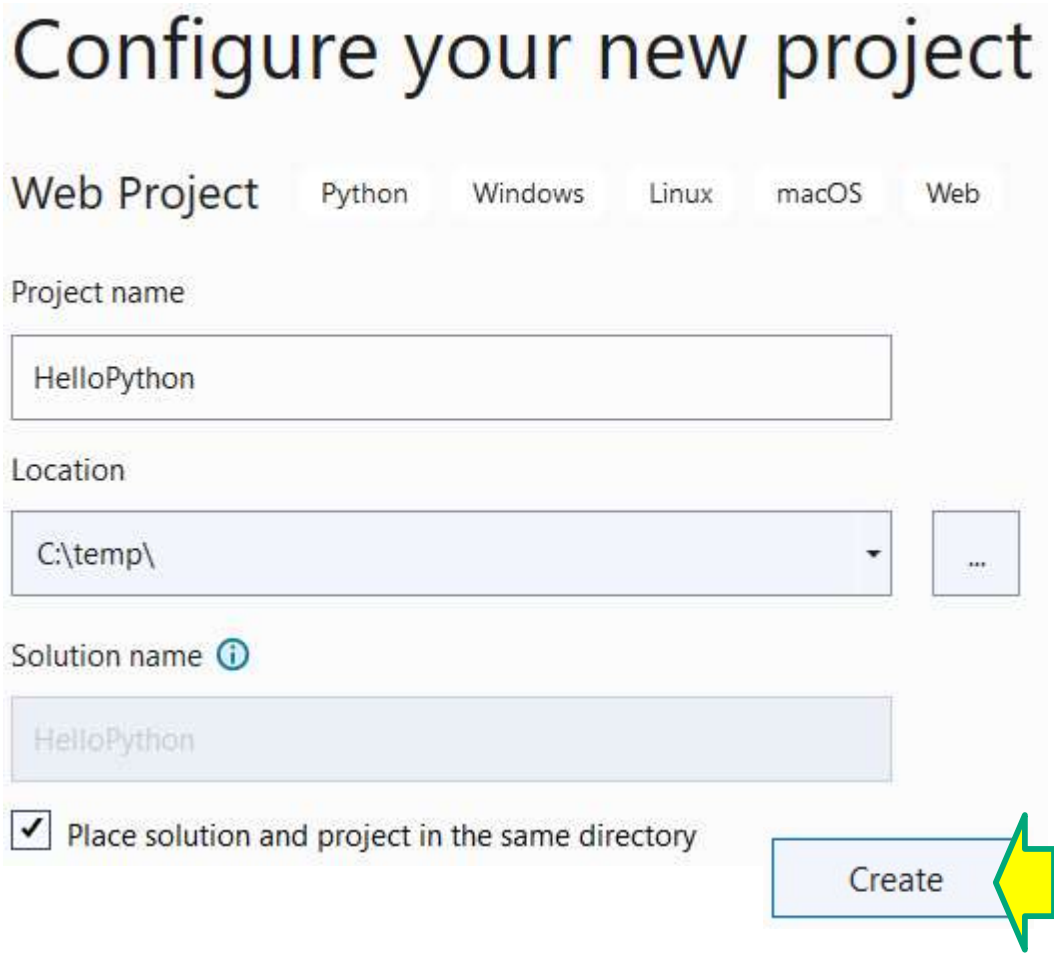


Create a New Project

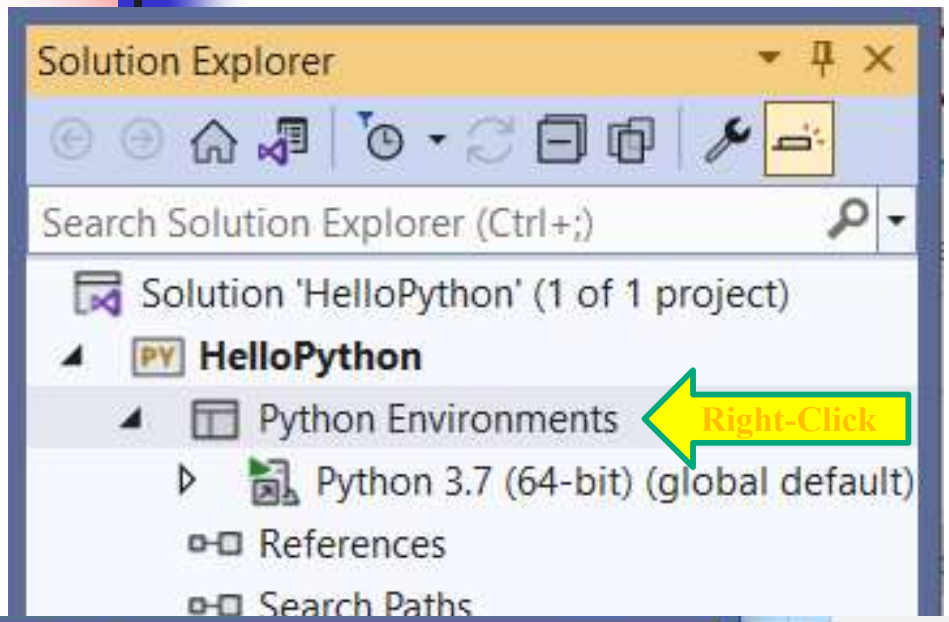
- Start Visual Studio →



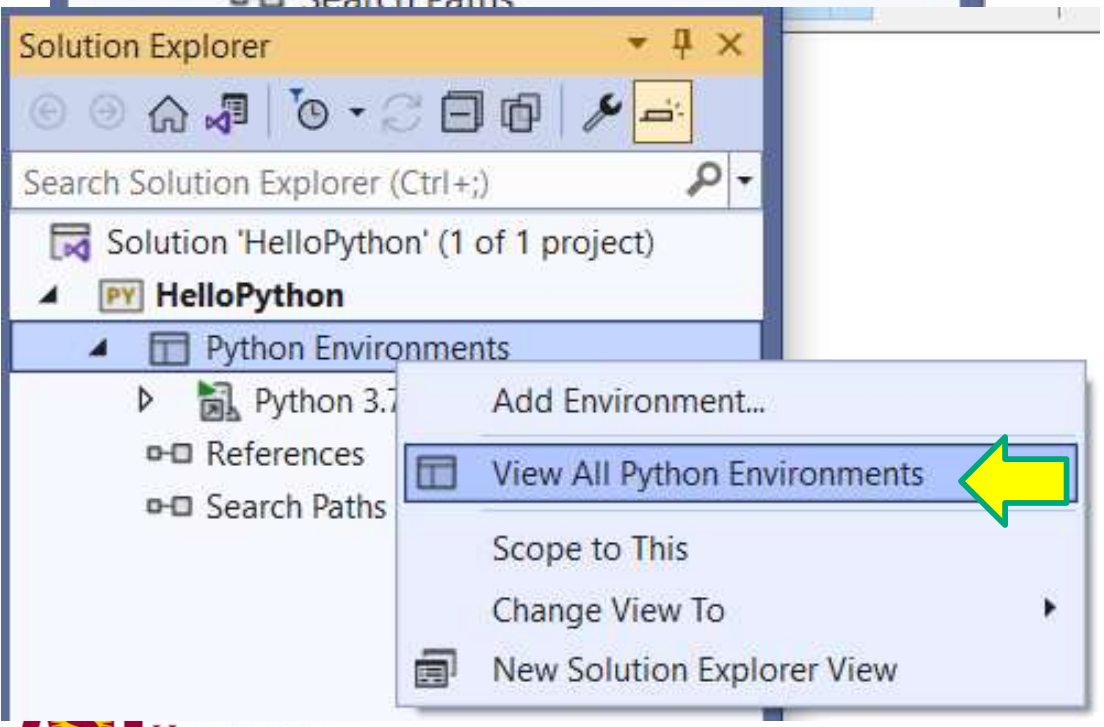


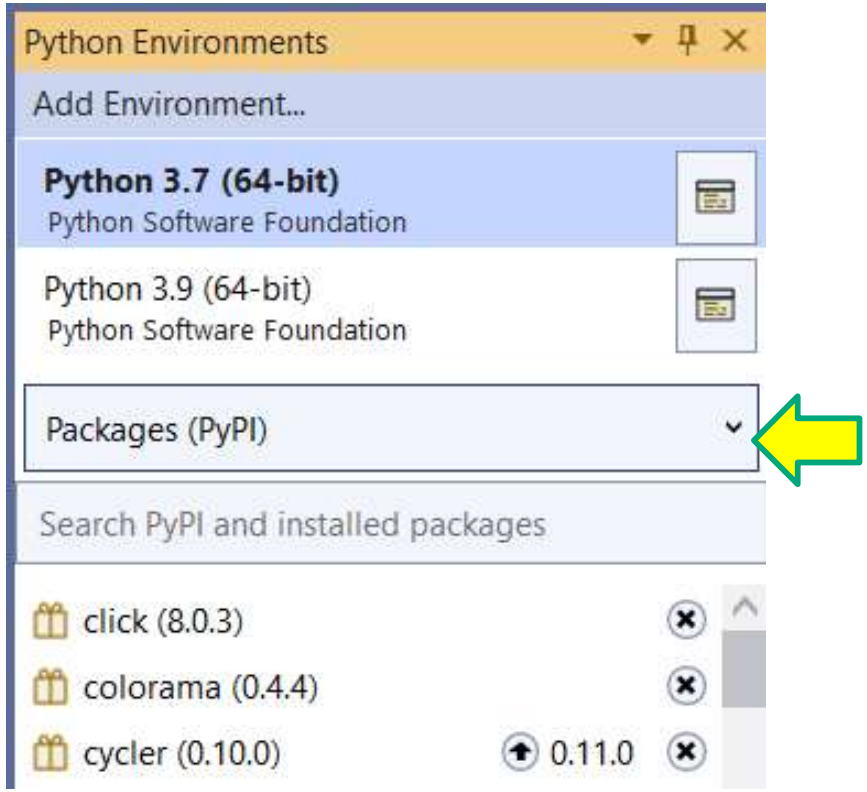


Project Created

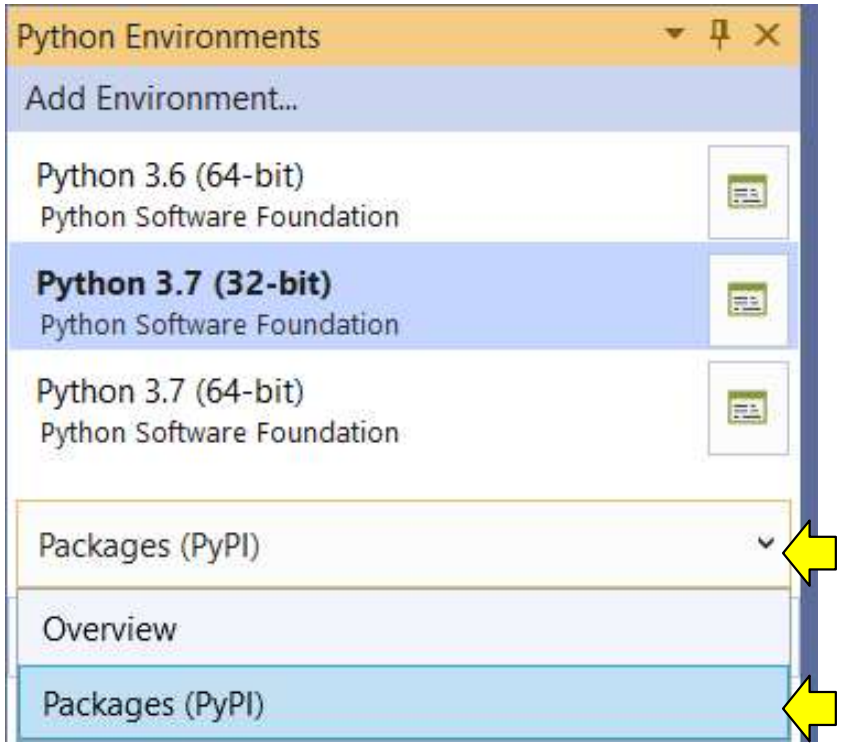


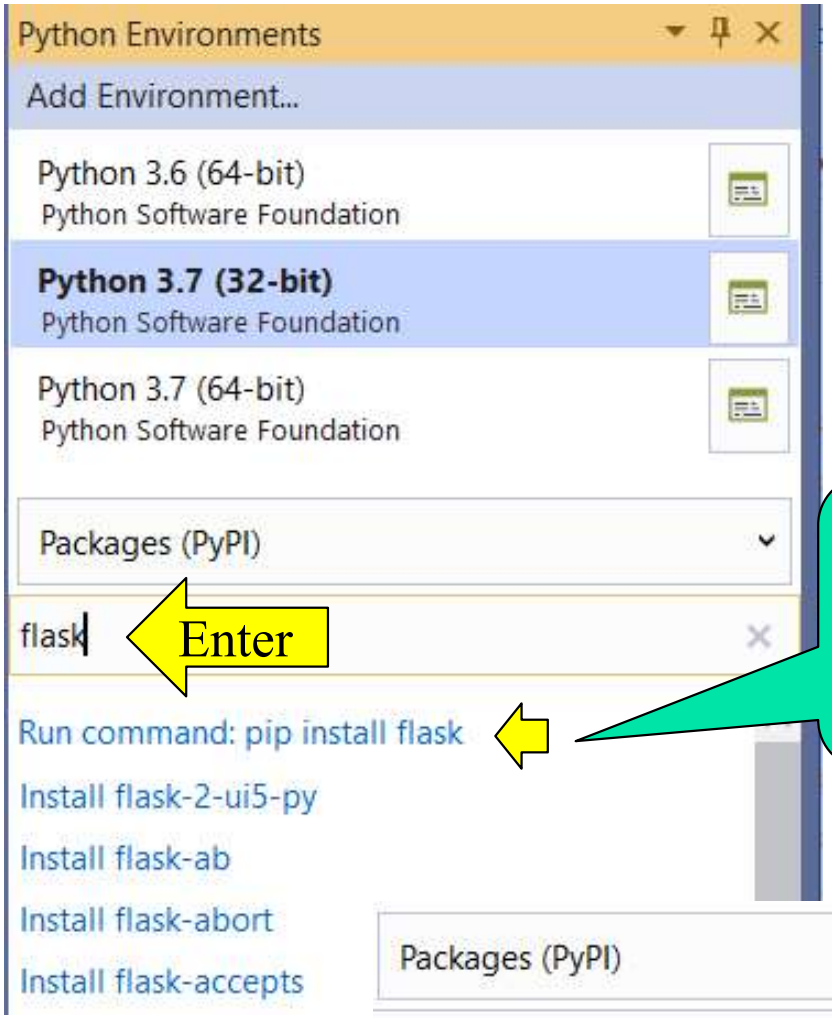
Right-click the environment and select **View All Python Environment**.



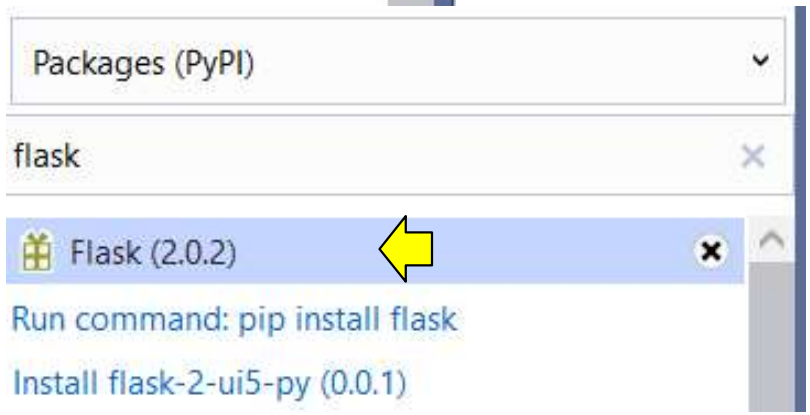


Install Flask Environment

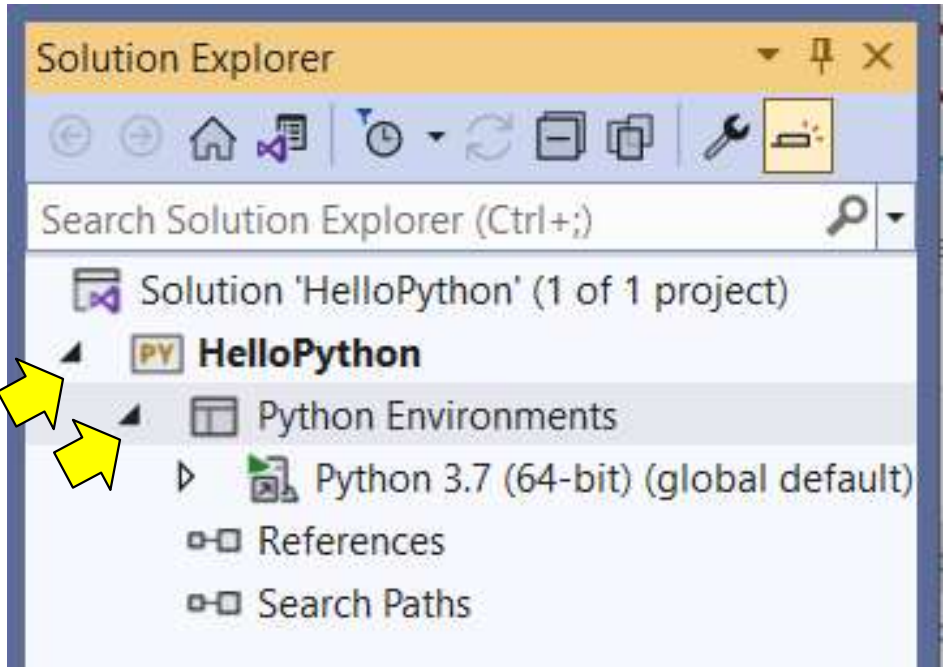


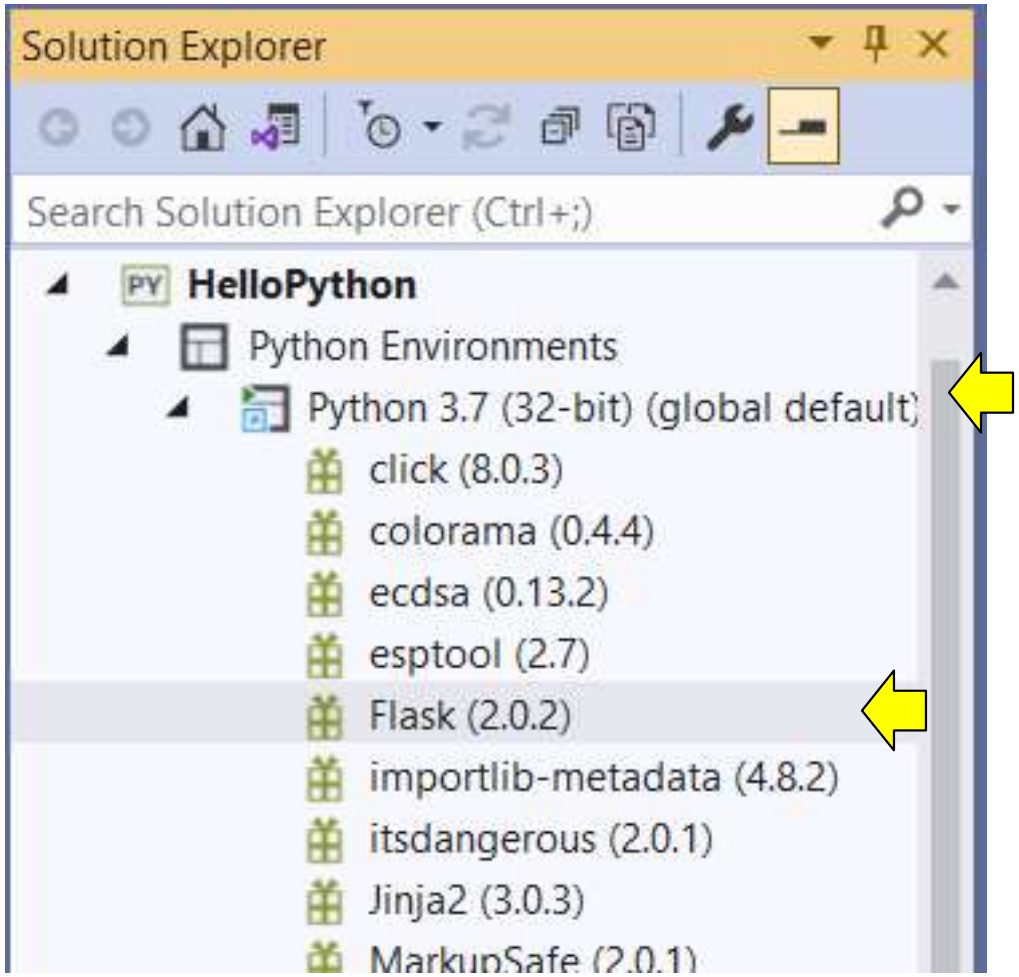


You may be asked to elevate the administer privilege. Elevate it.



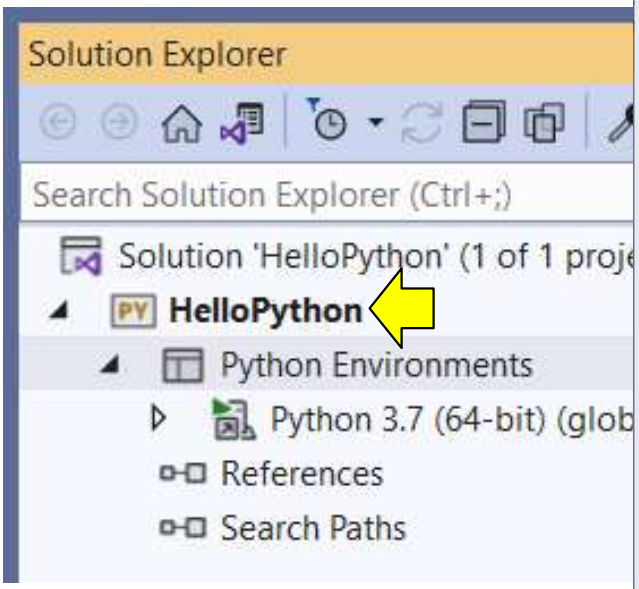
Return to Solution: Now, You should see Flask

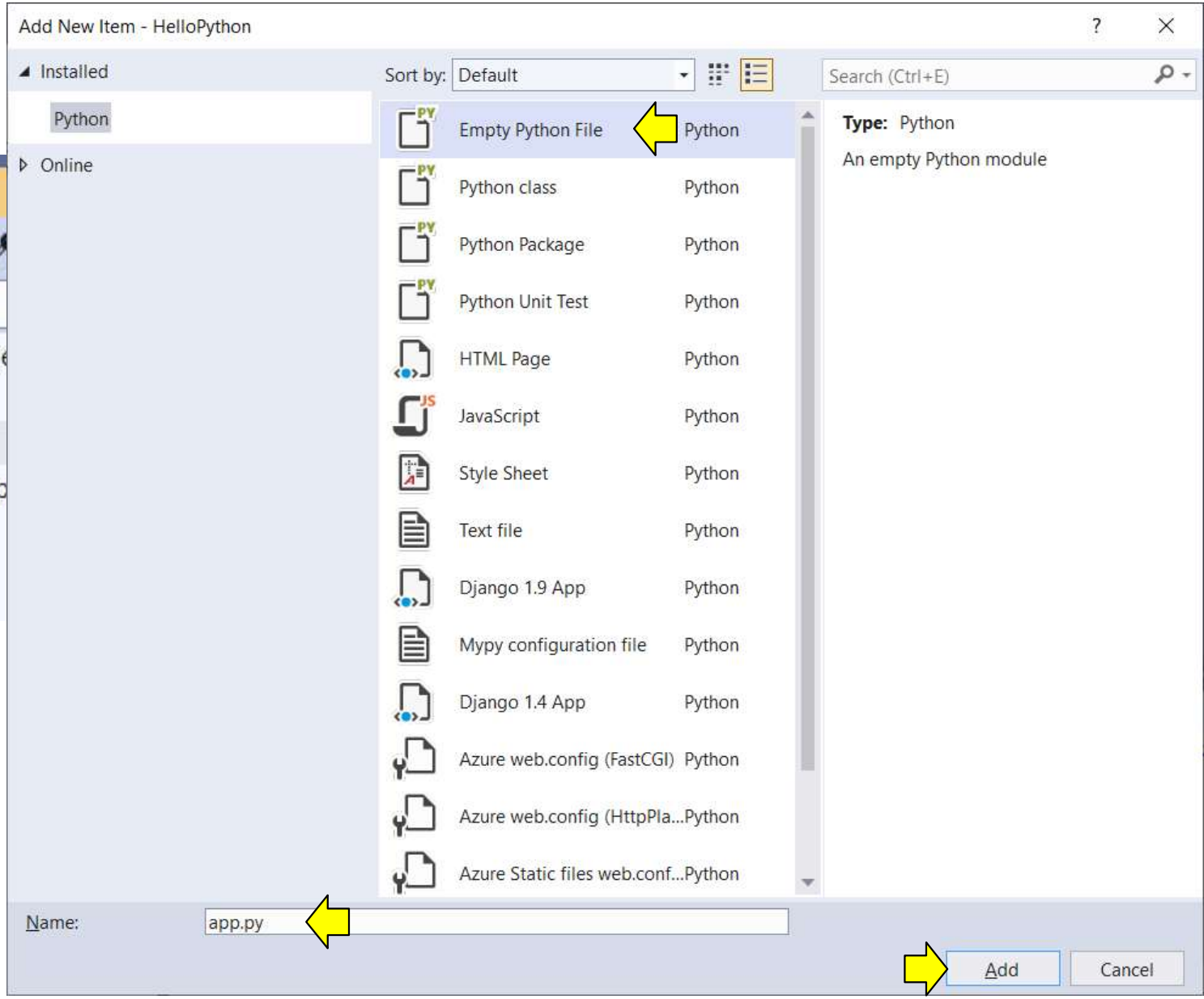




Add an Empty Python File

- Right-click Project name “HelloPython” and Choose Add New Item



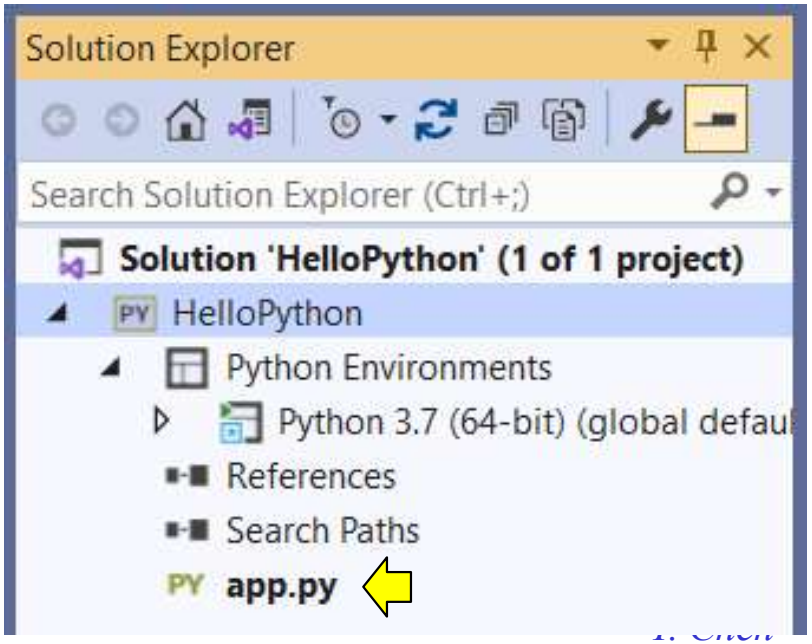


Enter the Service Code into app.py File

```
from flask import Flask
# Create an instance of the Flask class that is the WSGI application.
# The first argument is the name of the application module or
package,
# typically __name__ when using a single module.
app = Flask(__name__)
# Flask route decorators map / and /hello to the hello function.
# To add other resources, create functions that generate the page
contents
# and add decorators to define the appropriate resource locators for
them.
@app.route('/')
@app.route('/hello')
def hello():
    # Render the page
    return "Hello Python!"
if __name__ == '__main__':
    # Run the app server on localhost:4449
    app.run('localhost', 4449)
```

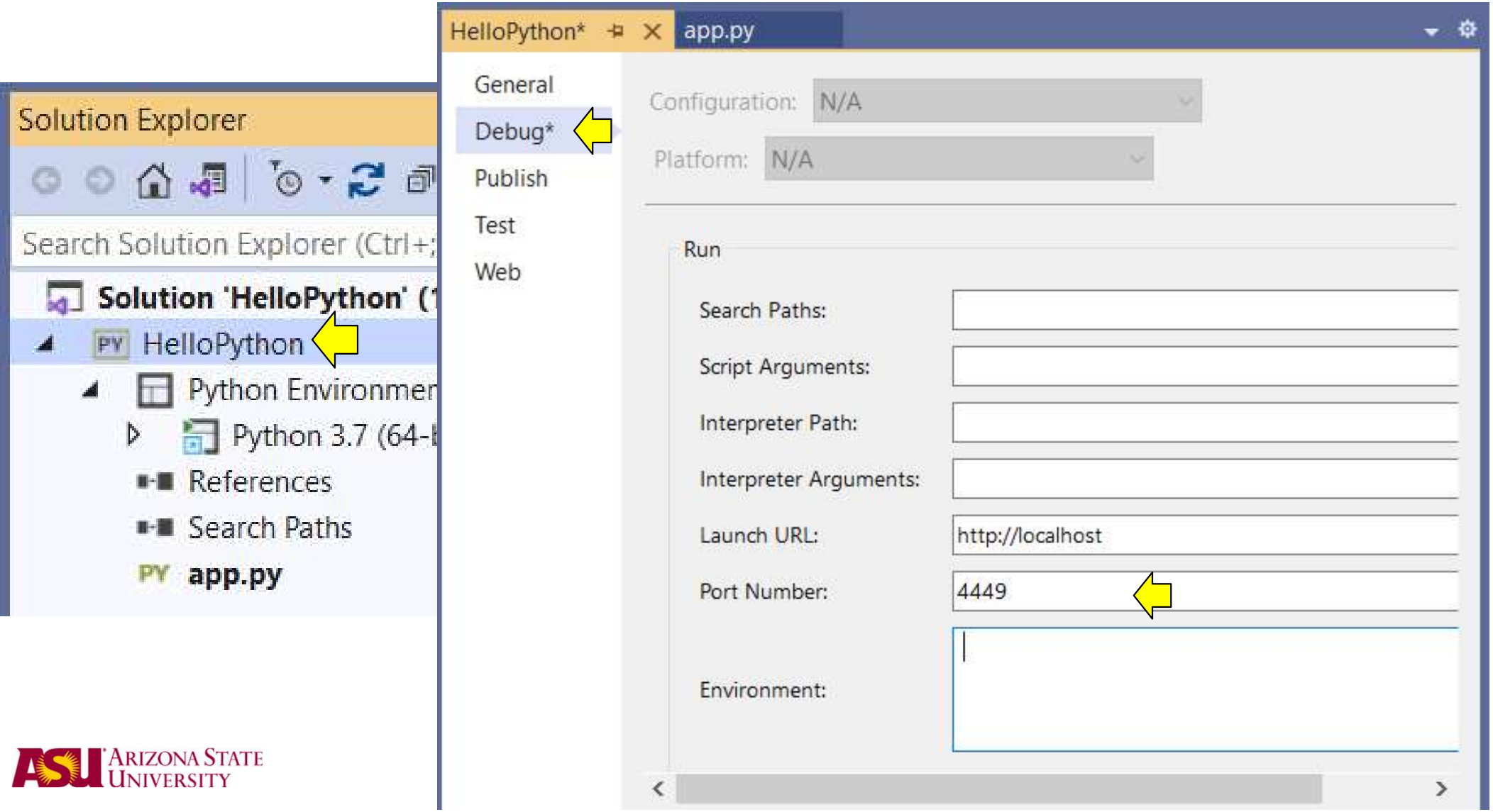
Service operation

Service hosting



Define Port Number in Visual Studio

Right-click the HelloPython project and select **Properties**. Select the **Debug** tab from the **Properties** menu, and Enter the **Port Number** property to 4449



Select Debug > Start Without Debugging

C:\windows\system32\cmd.exe

```
 * Serving Flask app 'app' (lazy loading)
 * Environment: production
   WARNING: This is a development server. Do not use it in a production deployment.
   Use a production WSGI server instead.
 * Debug mode: off
 * Running on http://localhost:4449/ (Press CTRL+C to quit)
127.0.0.1 - - [10/Dec/2021 16:09:00] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [10/Dec/2021 16:09:29] "GET /hello HTTP/1.1" 200 -
```

Open Web Browser

localhost:4449

Hello Python!

Service Code from the previous page

```
@app.route('/')
@app.route('/hello')
def hello():
    # Render the page
    return "Hello Python!"
if __name__ == '__main__':
    # Run the app server on lo
    app.run('localhost', 4449)
```

Hosting Python Service / Application on Wins Server

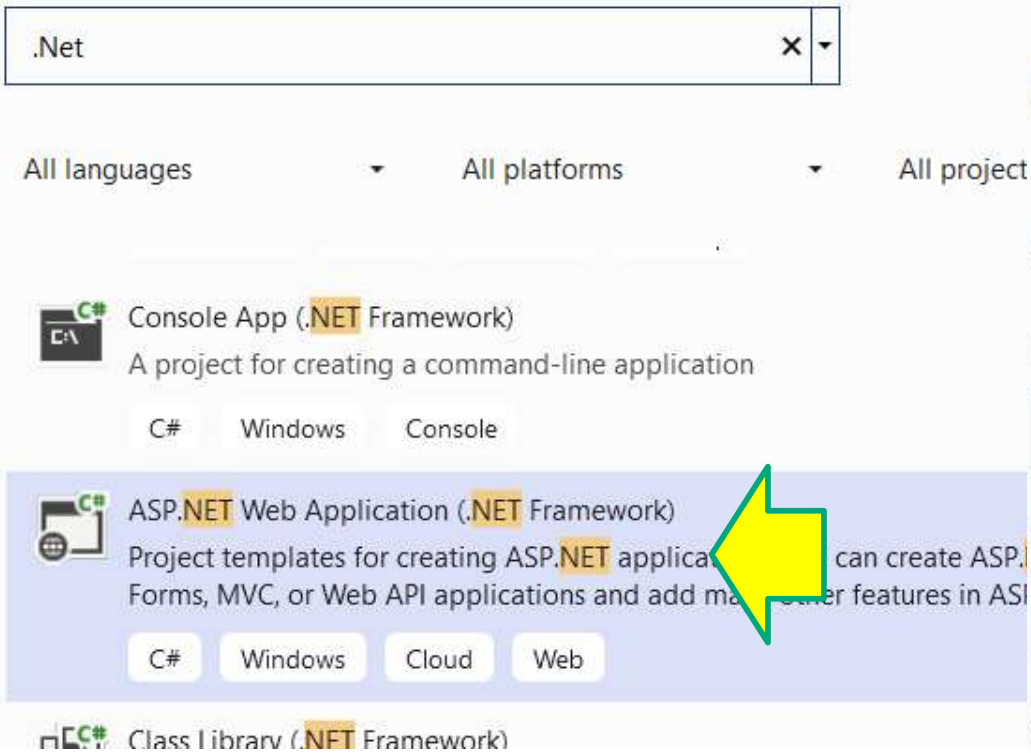
If your Web service/application is built with a non .Net language, you can create Windows containers to host them in Windows server or a cloud environment.

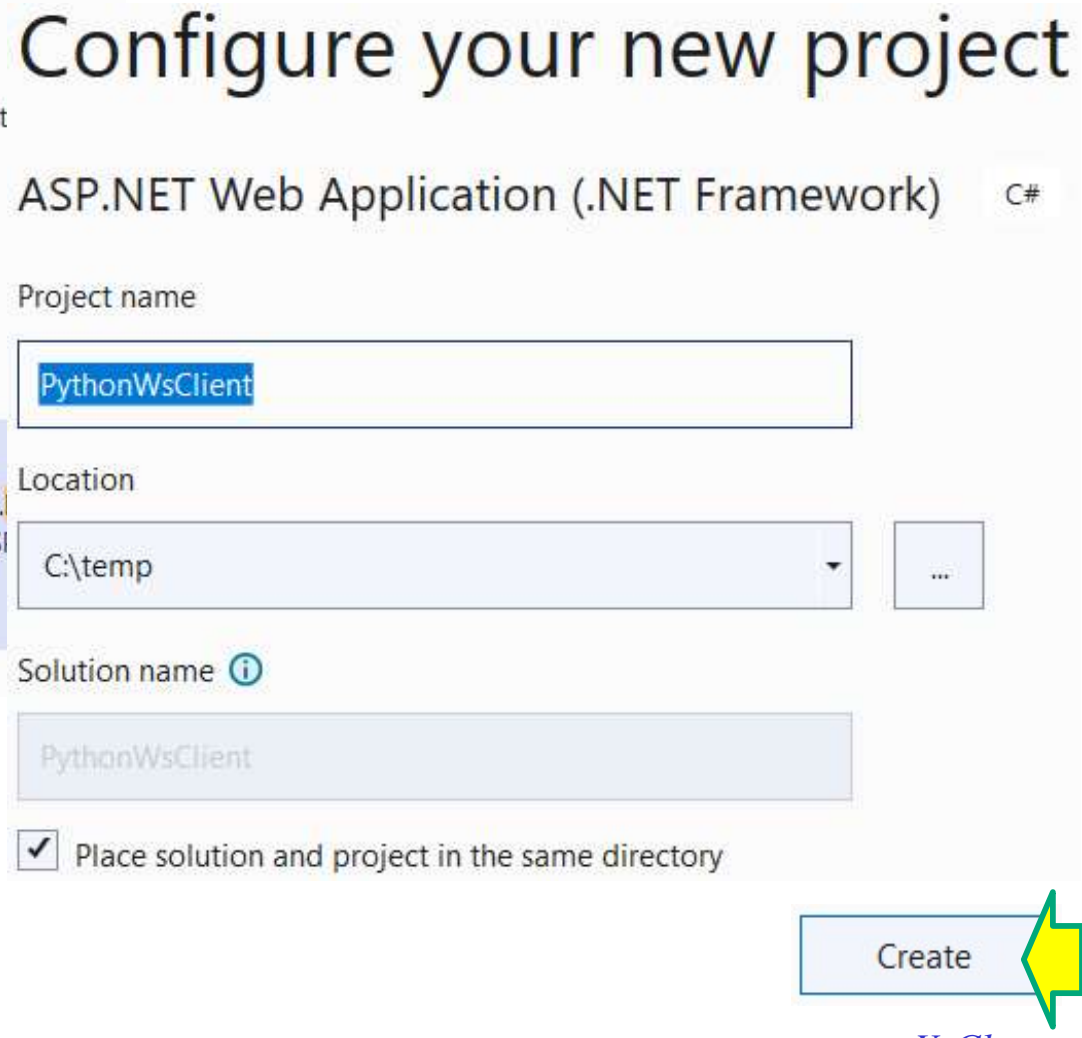
- Create Web application with Flask Web Project.
- To containerize your Web application, you will need to add a Docker file.
- Test your application locally
- Push your application to server and register the application.
- Full Tutorial: https://azure.github.io/AppService/2021/03/04/How-to-Host-a-Python-application-with-Windows-Containers-on-App-Service.html
- Example: https://venus.sod.asu.edu/WSRepository/repository.html

  Search Python

Create a New C# Project to Consume the Service

- Start Visual Studio →





Choose Empty Application

Create a new ASP.NET Web Application

Empty

An empty proje plate for creating ASP.NET applications. This template does not have any content in it

Web Forms

A project template for creating ASP.NET Web Forms applications. ASP.NET Web Forms lets you build dynam using a familiar drag-and-drop, event-driven model. A design surface and hundreds of controls and compo you rapidly build sophisticated, powerful UI-driven sites with data access.

MVC

A project template for creating ASP.NET MVC applications. ASP.NET MVC allows you to build applications u Model-View-Controller architecture. ASP.NET MVC includes many features that enable fast, test-driven deve for creating applications that use the latest standards.

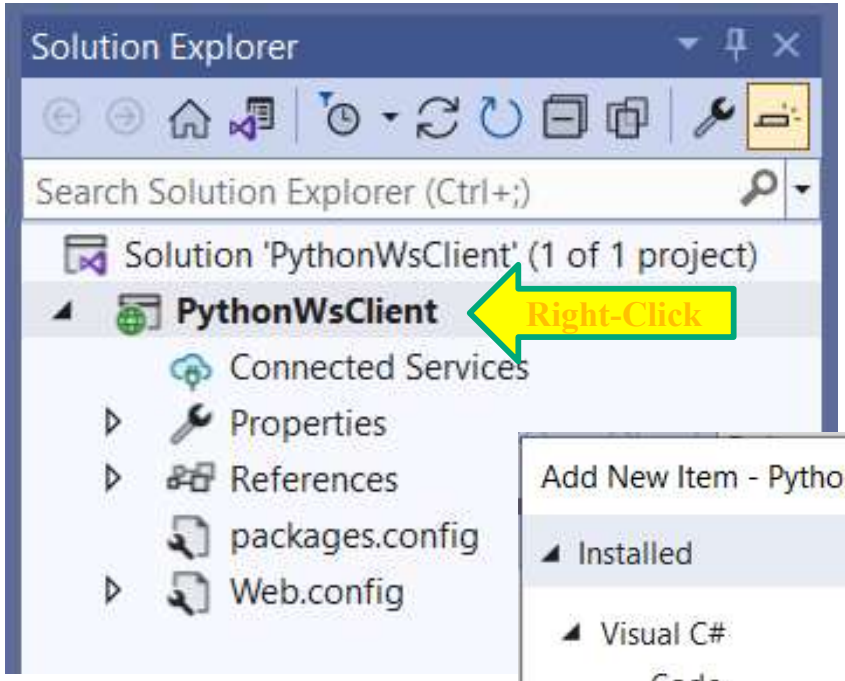
Web API

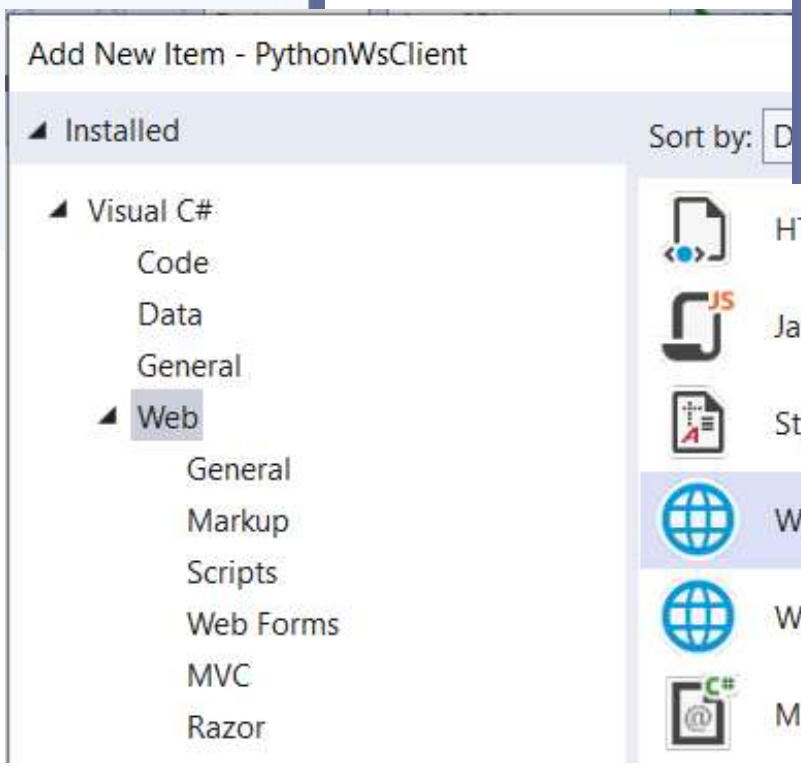
A project template for creating RESTful HTTP services that can reach a broad range of clients including brov mobile devices.

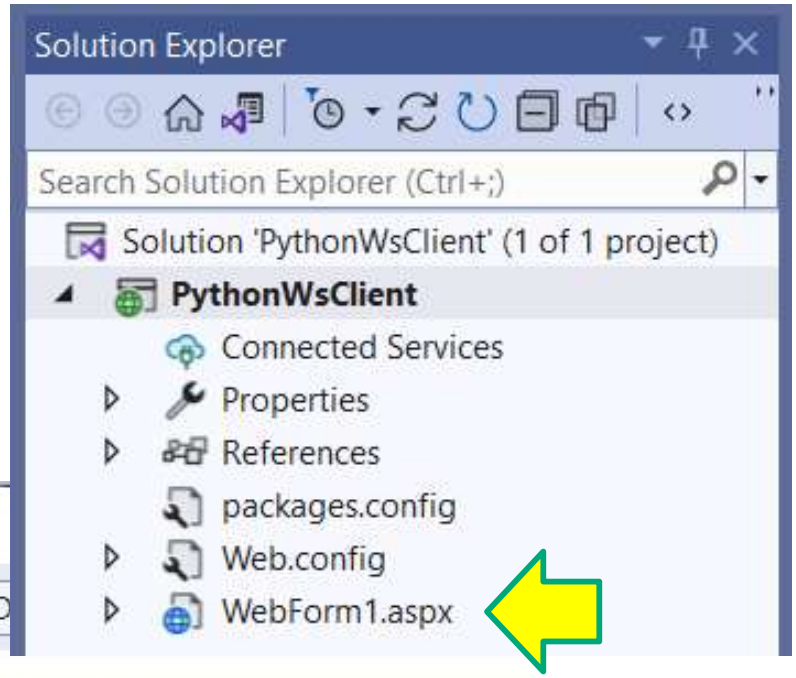
Create

Create A Client to Access Python Service

- Right-click Project name "PythonWsClient" and Choose Add New Item







Enter the C# Code to Access RESTful Service

WebForm1.aspx.cs | WebForm1.aspx | PythonWsClient: Connected Services

PythonWsClient | PythonWsClient.WebForm1 | txtUrl

```
using System;
using System.Net;

namespace PythonWsClient
{
    1 reference
    public partial class WebForm1 : System.Web.UI.Page
    {
        0 references
        protected void Page_Load(object sender, EventArgs e)
        {
        }

        0 references
        protected void btnInvoke_Click(object sender, EventArgs e)
        {
            Uri completeUri = new Uri(txtUrl.Text);
            WebClient channel = new WebClient(); // create a channel
            byte[] abc = channel.DownloadData(completeUri); // return byte arra
            string str = System.Text.Encoding.Default.GetString(abc);// convent
            lblResult.Text = str;
        }
    }
}
```

Solution Explorer

Search Solution Explorer (Ctrl+;)

- Solution 'PythonWsClient' (1 of 1 project
- PythonWsClient
  - Connected Services
  - Properties
  - References
  - Web.config
  - WebForm1.aspx
    - WebForm1.aspx.cs
    - WebForm1.aspx.designer.cs

Solution Explorer | Team Explorer

Properties

Accessing Python Service in C# Client Code

```
using System;
using System.Net; // Contain WebClient class
namespace PythonWsClient
{
    public partial class WebForm1 : System.Web.UI.Page
    {
        protected void Page_Load(object sender, EventArgs e)
        {
        }
        protected void btnInvoke_Click(object sender, EventArgs e)
        {
            Uri completeUri = new Uri(txtUrl.Text); //http//localhost/4449
            WebClient channel = new WebClient(); // create a channel
            byte[] abc = channel.DownloadData(completeUri);
               // return byte array
            string str = System.Text.Encoding.Default.GetString(abc);
               // convent to string
            lblResult.Text = str;
        }
    }
}
```

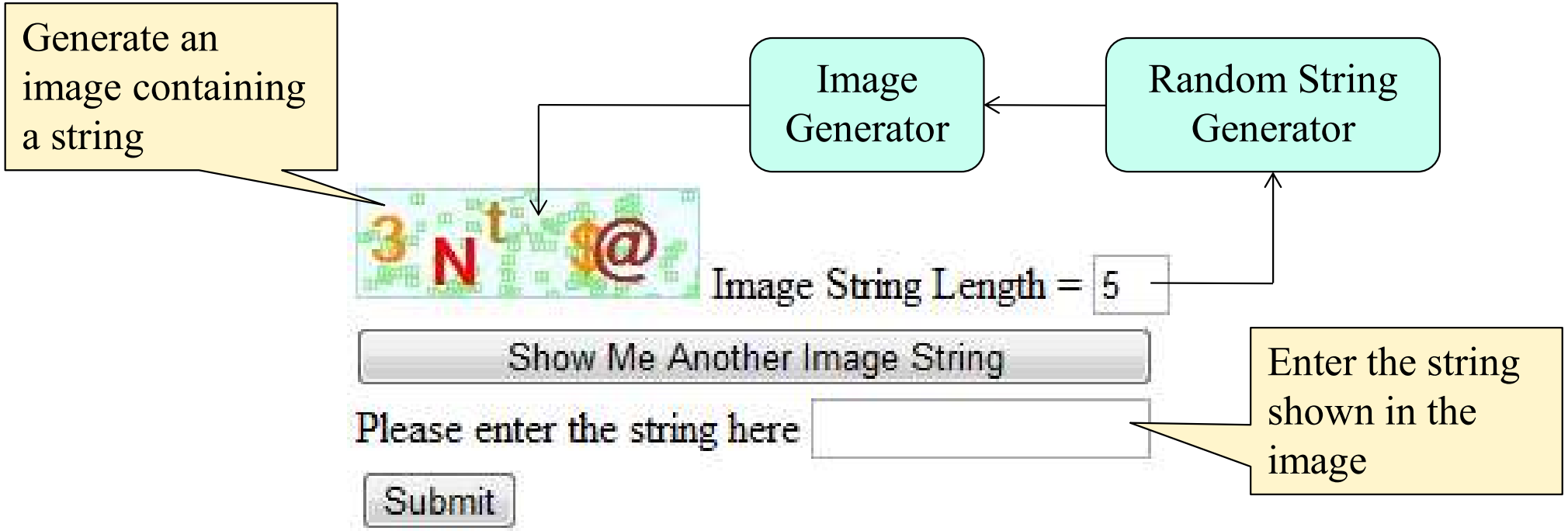
The same code that consume a C# service

Lecture Roadmap: Case Study 1

- **Developing REST Services:**
  - Developing RESTful Service
  - Defining RESTful Service in Python
- **Image Verifier in RESTful Service**
  - RESTful Service of a Random String Generator
  - RESTful Service of an Image Verifier
  - TryIt page calling RESTful Services
- RESTful services in ASP .Net Core

Development of an Image Verifier

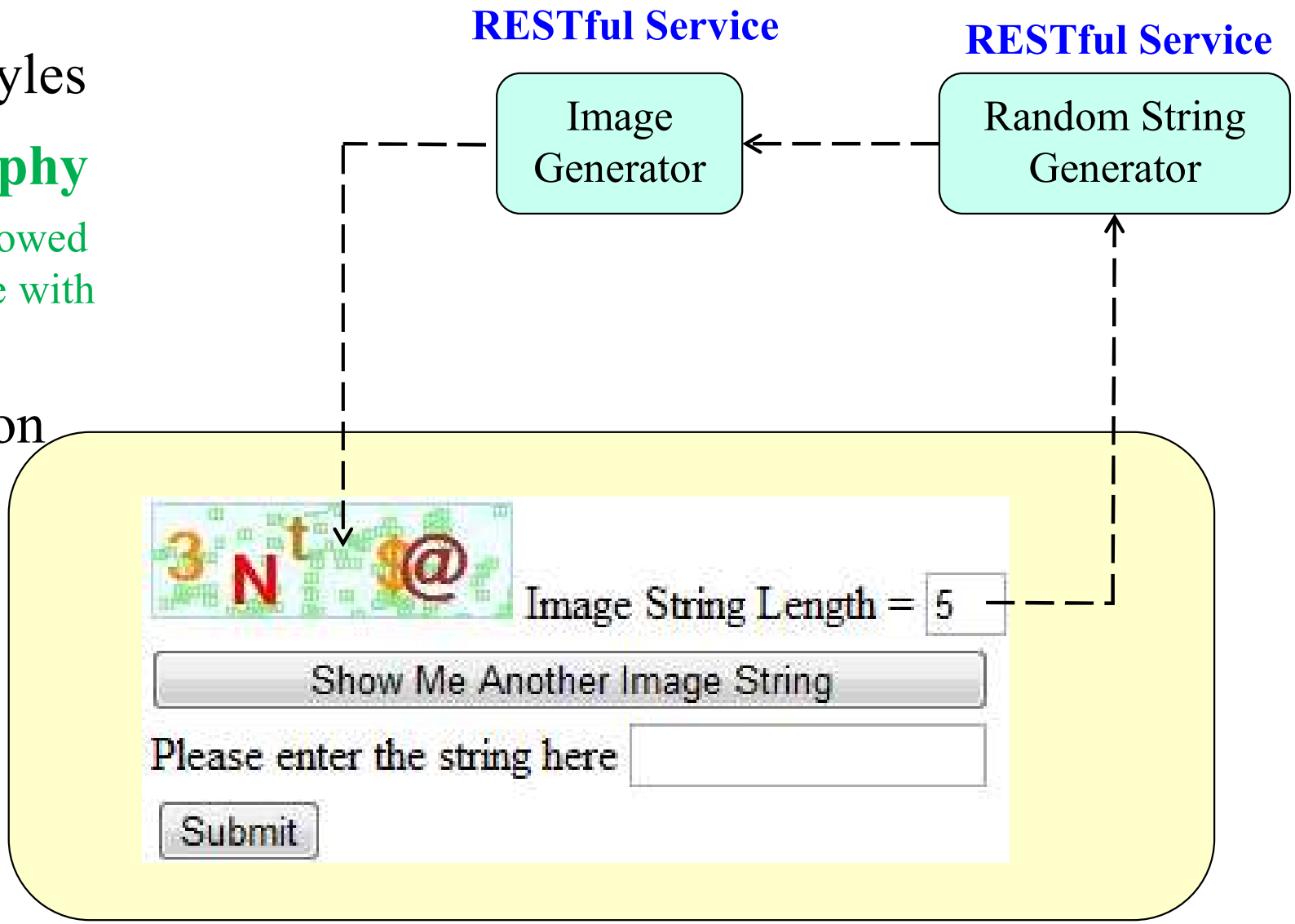
- Image verifier (CAPTCHA) is widely used as a way of preventing programmed attached to Web sites.
- An image verifier consists of a random string generator and an image generator.



Architecture Design of the Image Verifier

Composition Architecture Styles

- ☑ **Choreography**
  Services are allowed to communicate with each other
- ☐ Orchestration

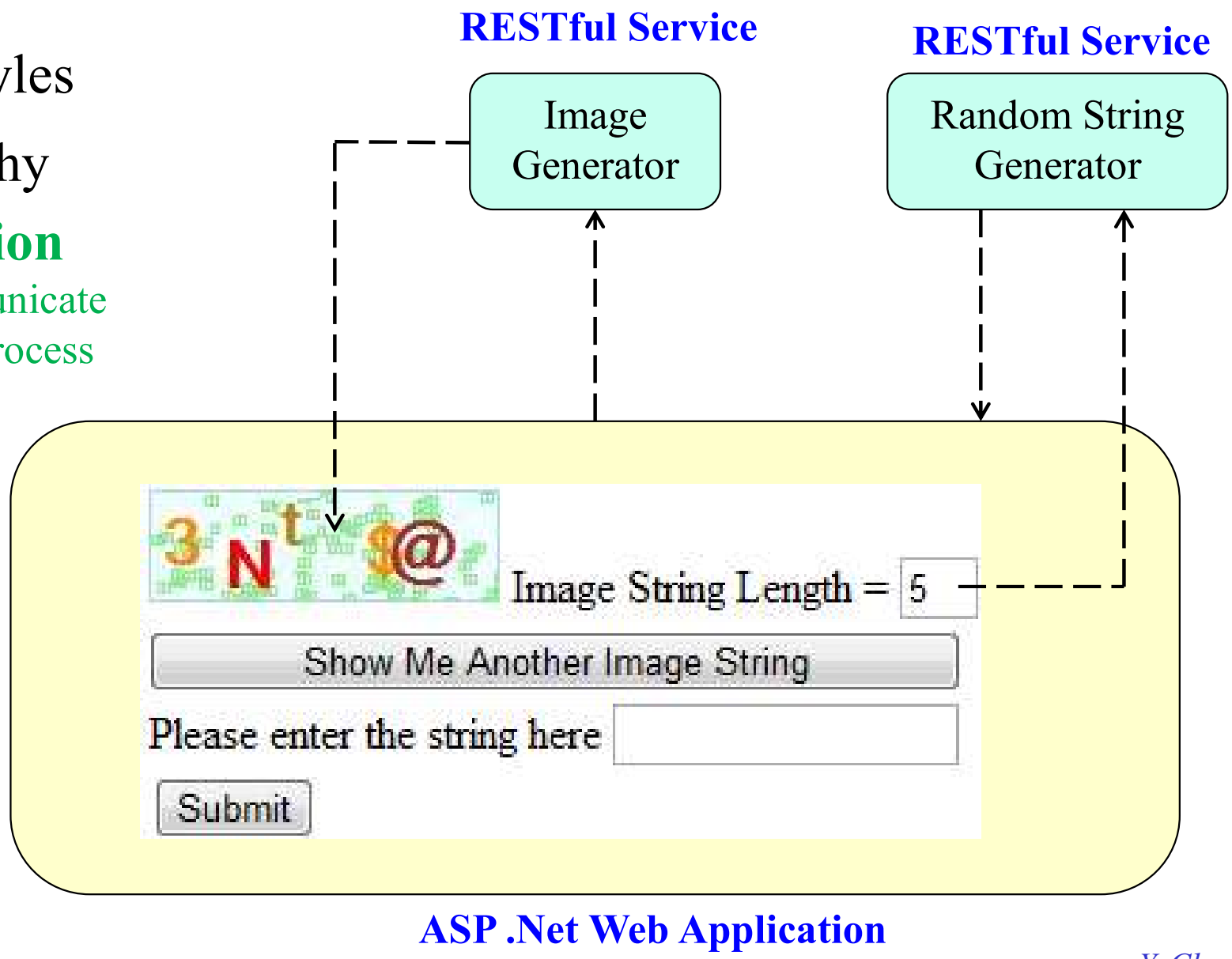


ASP .Net Web Application

Architecture Design of the Image Verifier

Composition Architecture Styles

- ☐ Choreography
- ☑ **Orchestration**
  Services communicate with a central process only



Lecture Roadmap

- **Developing REST Services:**
  - Developing RESTful Service
  - Defining RESTful Service in Python
- **Image Verifier in RESTful Service**
  - RESTful Service of a Random String Generator
  - RESTful Service of an Image Verifier
  - Synchronous RESTful Service Calls
- RESTful services in ASP .Net Core

Example: Random String Generator

- A RESTful service deployed at:

http://venus.sod.asu.edu/WSRepository/Services/RandomString/Service.svc/

- It has two operations:
  - Take an integer (length) as input, it returns a random string of the given length
  - Without providing a parameter, it returns a random string with a random length between 4 and 20.
- The random string will contain an uppercase letter, a lowercase letter, a digit, and a special character. It can be used as a strong password.
- They can be accessed in URI:

http://venus.sod.asu.edu/WSRepository/Services/RandomString/Service.svc/GetRandomString/5
http://venus.sod.asu.edu/WSRepository/Services/RandomString/Service.svc/GetRandomString

Random String Generator: IService.cs

We are using the style with svc endpoints created but disabled

```
using System; using System.ServiceModel;
using System.ServiceModel.Web;
[ServiceContract]
public interface IService {
    [WebGet(UriTemplate = "/GetRandomString", RequestFormat =
    WebMessageFormat.Xml, ResponseFormat =
    WebMessageFormat.Xml, BodyStyle =
    WebMessageBodyStyle.Bare)]
    string GetRandomString0();

    [WebGet(UriTemplate = "/GetRandomString/{Length}",
    RequestFormat = WebMessageFormat.Xml, ResponseFormat =
    WebMessageFormat.Xml, BodyStyle =
    WebMessageBodyStyle.Bare)]
    string GetRandomString(string Length);
}
```

Names must be different: cannot overload

Names to be used in URI can be the same

Lecture Roadmap

- **Developing REST Services:**
  - Developing RESTful Service
  - Defining RESTful Service in Python
- **Image Verifier in RESTful Service**
  - RESTful Service of a Random String Generator
  - RESTful Service of an Image Verifier
  - TryIt page calling RESTful Services
- RESTful services in ASP .Net Core

Image Verifier Interface: IService.cs

```
using System; using System.IO;
using System.ServiceModel;
using System.ServiceModel.Web;
[ServiceContract]
public interface IService {
    [WebGet(UriTemplate = "/GetVerifierString/{myLength}",
    RequestFormat = WebMessageFormat.Xml, ResponseFormat =
    WebMessageFormat.Xml, BodyStyle = WebMessageBodyStyle.Bare)]
    string GetVerifierString(string myLength); // generates a string

    [WebGet(UriTemplate = "/GetImage/{myString}", RequestFormat =
    WebMessageFormat.Xml, ResponseFormat =
    WebMessageFormat.Xml, BodyStyle = WebMessageBodyStyle.Bare)]
    Stream GetImage(string myString); // generates image of the string
}
```

This operation will call the RandomString/Service.svc

Image Verifier: Service.cs Architecture

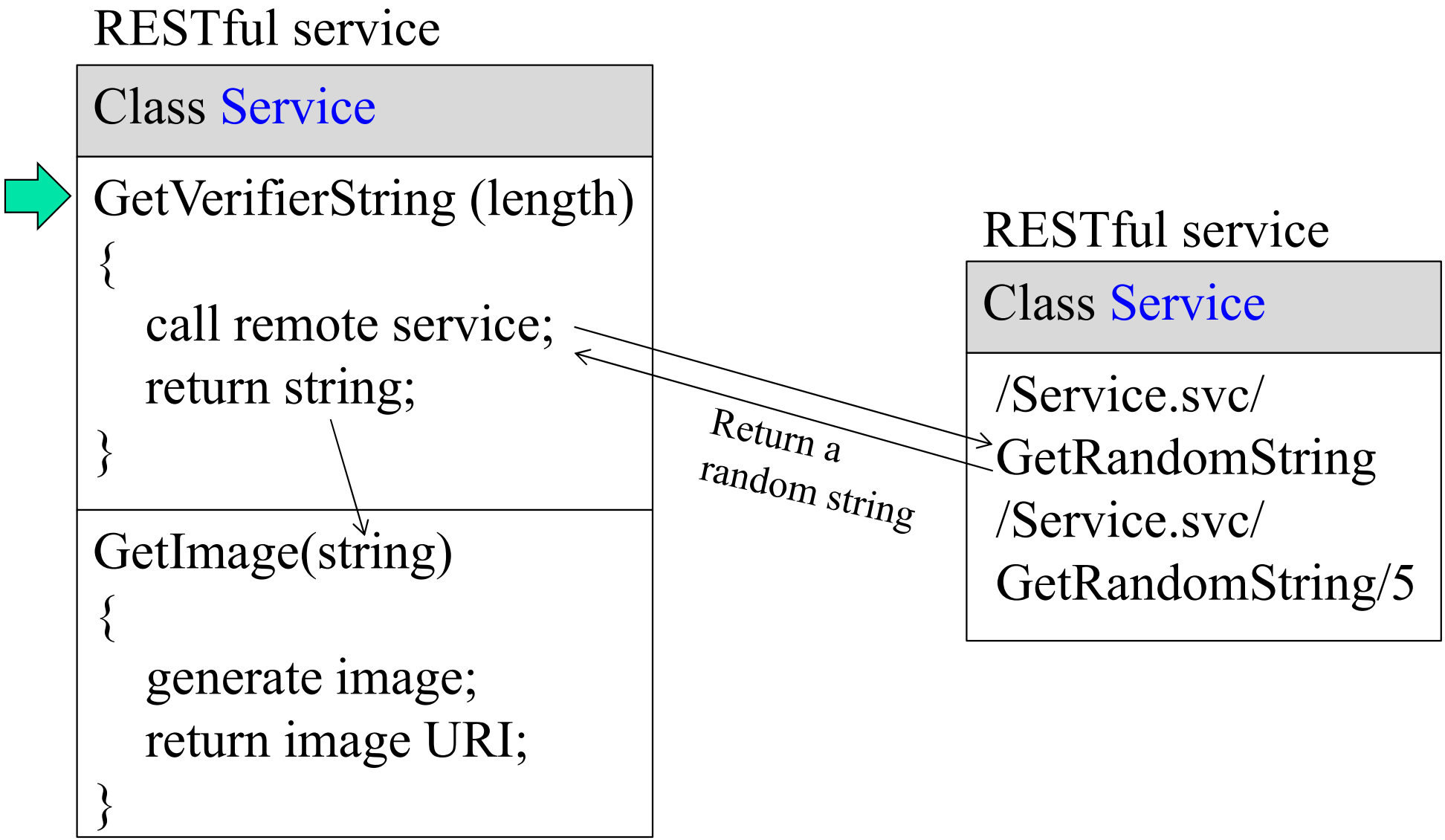


Image Verifier: Service.cs Code (1)

Notice how a RESTful service is access here

```
public class Service : IService {
    public string GetVerifierString(string myLength) {
        // Create the base address to the RandomString service
1       Uri baseUri = new
        Uri("http://venus.sod.asu.edu/WSRepository/Services/RandomString/Service.svc");
        // Define UriTemplate for passing parameter
2       UriTemplate myTemplate = new UriTemplate("GetRandomString/{Length}");
        // Assign values to variable to obtain the complete URI
3       Uri completeUri = myTemplate.BindByPosition(baseUri, myLength);
4       WebClient channel = new WebClient(); // create a channel
5       byte[] abc = channel.DownloadData(completeUri); // return byte array
6       Stream strm = new MemoryStream(abc); // convert to mem stream
7       DataContractSerializer obj = new DataContractSerializer(typeof(string));
8       string randString = obj.ReadObject(strm).ToString();
9       return randString;
}
```

The purpose is to create a language-independent string

Image Verifier: Service.cs Code (1) Explained

The service to be called:

http://venus.sod.asu.edu/WSRepository/Services/RandomString/Service.svc/GetRandomString/5

From the previous page:

```
1   // Create the base address to the RandomString service
    Uri baseUri = new Uri("http://venus.sod.asu.edu/WSRepository/Services/RandomString/Service.svc");
2   // Create the Template to be used by the client
    UriTemplate myTemplate = new UriTemplate("GetRandomString/{Length}");
3   // Assign values to variable to obtain the complete URI
    Uri completeUri = myTemplate.BindByPosition(baseUri, myLength);
```

- Can we use string appending, instead?

```
Uri completeUri = baseUri + myLength;
```

Image Verifier: Service.cs Architecture

RESTful service

Class Service

```
GetVerifierString (length)
{
    call remote service;
    return string;
}
```

```
GetImage(string)
{
    generate image;
    return image URI;
}
```

Return a random string

RESTful service

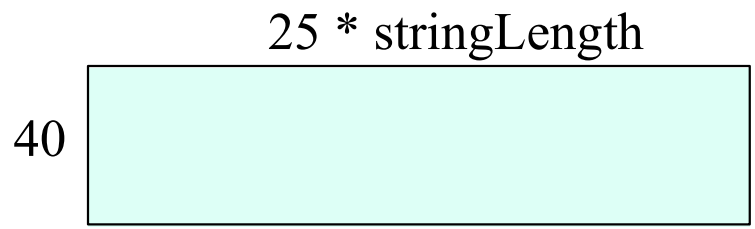
Class Service

/Service.svc/GetRandomString

/Service.svc/GetRandomString/5

http://venus.sod.asu.edu/WSRepository/Services/ImageVerifier/Service.svc/GetImage/7Mt$@

Image Verifier: Service.cs Code (2)




```
// This operation creates image based on the input string
public Stream GetImage(string myString) {
    WebOperationContext.Current.OutgoingResponse.ContentType =
"image/jpeg";
    int mapwidth = (int)(myString.Length * 25);   // define bitmap width
    Bitmap bMap = new Bitmap(mapwidth, 40); // based on string length
    Graphics graph = Graphics.FromImage(bMap);
    graph.Clear(Color.Azure);                          // background color
    graph.DrawRectangle(new Pen(Color.LightBlue, 0), 0, 0, bMap.Width - 1,
bMap.Height - 1); // draw a frame
```

We have two variables: bMap and graph.
We use graph variable to modify bMap.

Image Verifier: Service.cs Code (3)

We continue to use graph to modify bMap variable.

```
Random rand = new Random();
Pen badPen = new Pen(Color.LightGreen, 0);
for (int i = 0; i < 100; i++) {    // create random noise pattern
    int x = rand.Next(1, bMap.Width - 1);
    int y = rand.Next(1, bMap.Height - 1);
    graph.DrawRectangle(badPen, x, y, 4, 3);
    graph.DrawEllipse(badPen, x, y, 2, 3);  // position and size
}
char[] charString = myString.ToCharArray();
Font font = new Font("Boopee", 18, FontStyle.Bold);
Color[] clr = {Color.Black, Color.Red, Color.DarkViolet, Color.Green,
Color.DarkOrange, Color.Brown, Color.DarkGoldenrod, Color.Plum };
```

Define an array of different colors

Image Verifier: Service.cs Code (4)

```
// Draw the characters in the string onto the graphics object
for (int i = 0; i < myString.Length; i++) {
    int d = rand.Next(20, 25); // distance between characters
    int p = rand.Next(1, 15); // up and down position
    int c = rand.Next(0, 7); // randomly choose a color
    string str1 = Convert.ToString(charString[i]); //char->string
    Brush b = new System.Drawing.SolidBrush(clr[c]);
    graph.DrawString(str1, font, b, 1 + i * d, p);
}
System.IO.MemoryStream ms = new System.IO.MemoryStream();
bMap.Save(ms, System.Drawing.Imaging.ImageFormat.Jpeg);
ms.Position = 0;
graph.Dispose(); bMap.Dispose();
return ms;  // return the image generated
}
}
```

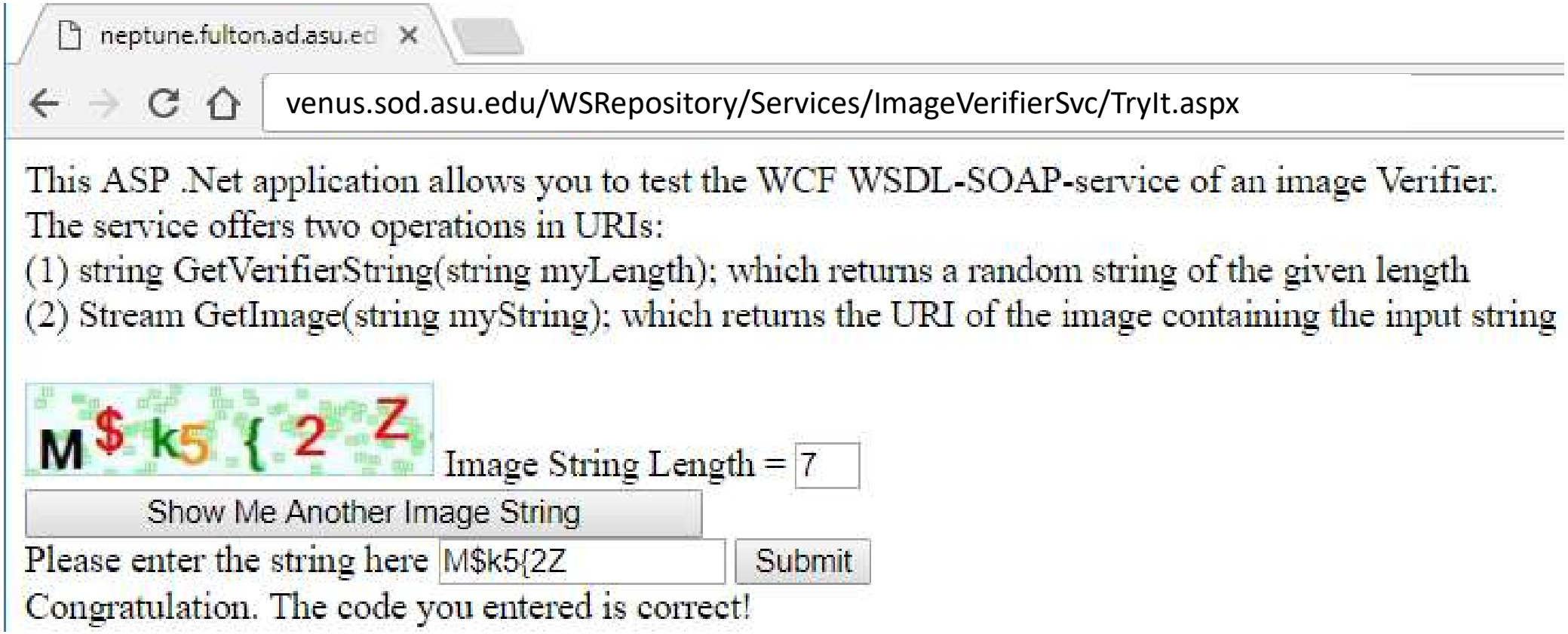
Now, we save bMap into memory stream variable ms

Lecture Roadmap

- **Developing REST Services:**
  - Developing RESTful Service
  - Defining RESTful Service in Python
- **Image Verifier in RESTful Service**
  - RESTful Service of a Random String Generator
  - RESTful Service of an Image Verifier
  - TryIt page calling RESTful Services
- RESTful services and Applications in ASP .Net Core

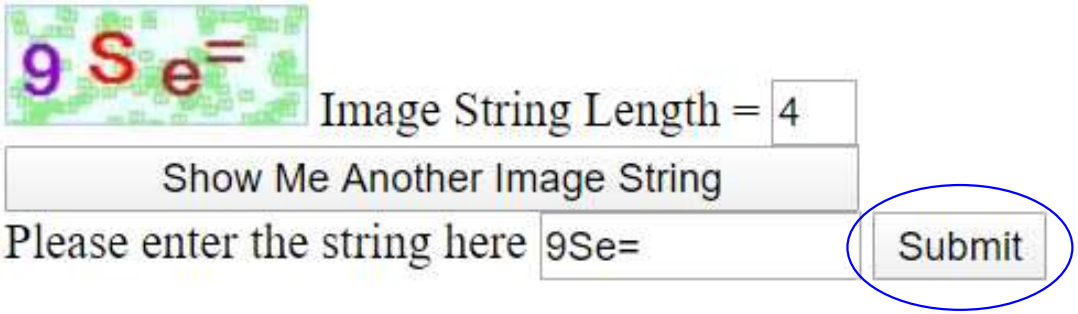
GUI Design and Code Behind GUI

http://venus.sod.asu.edu/WSRepository/Services/ImageVerifierSvc/TryIt.aspx



Code Behind the GUI: TryIt.aspx.cs (1)

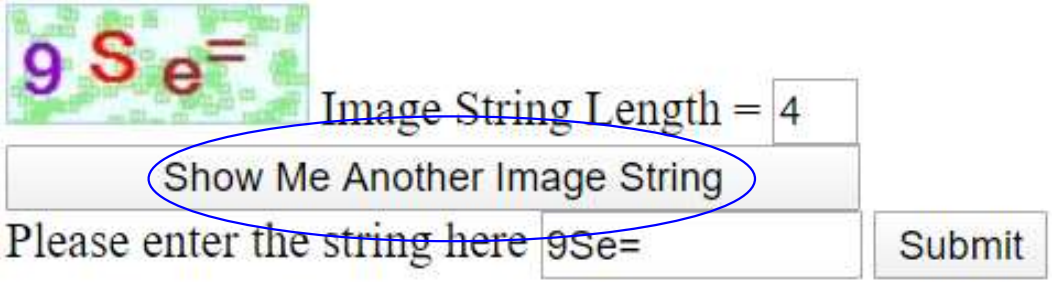
```
using System; using System.Net; using System.IO;
using System.Runtime.Serialization;
public partial class TestVerifier : System.Web.UI.Page {
    protected void Page_Load(object sender, EventArgs e) { }
    protected void btnVerifyImage_Click(object sender, EventArgs e) {
        if (Session["generatedStr"].Equals(TextBox1.Text)) {
            Label1.Text =
                        "Congratulation. The code you entered is correct!";
        }
        Else {
            Label1.Text = "I am sorry, the string you entered does not match
the image. Please try again!";
        }
    }
```



Code Behind the GUI: TryIt.aspx.cs (2)

```
protected void btnGetImage_Click(object sender, EventArgs e) {
    // create the base address
    Uri baseUri = new
    Uri("http://venus.sod.asu.edu/WSRepository/Services/ImageVerifier/Service.svc/");
    // define operation name and parameter template
    UriTemplate myTemplate = new
                                    UriTemplate("GetVerifierString/{myLength}");
    // assign values to variable to complete URI
    Uri completeUri = myTemplate.BindByPosition(baseUri,
                                                    txtBoxlength.Text);
```

Prepare the URI to call the GetImage service

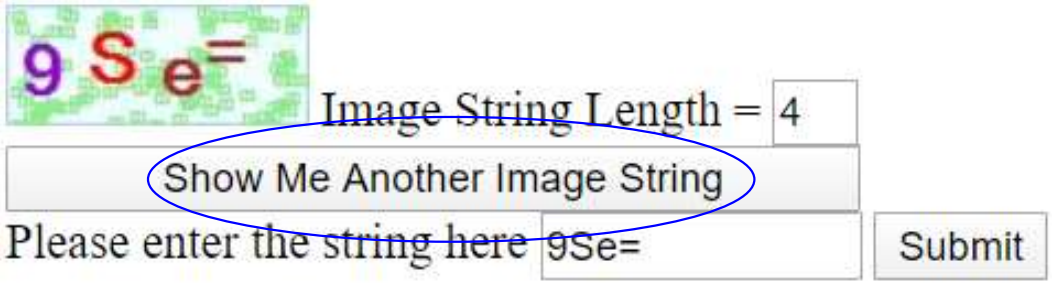


Code Behind the GUI: TryIt.aspx.cs (3)

call RESTful service GetVerifierString

```
WebClient channel = new WebClient();
byte[] abc = channel.DownloadData(completeUri);
Stream strm = new MemoryStream(abc);
DataContractSerializer obj = new
                                    DataContractSerializer(typeof(string));
string generatedString = obj.ReadObject(strm).ToString();
Session["generatedStr"] = generatedString; // Save
Image1.Visible = true;
Image1.ImageUrl =
  "http://venus.sod.asu.edu/WSRepository/Services/ImageVerifier/Service.svc/GetImage/" + generatedString;
btnGetImage.Text = "Show Me Another Image String";
    }
}
```

call the service GetImage using string as parameter



Read textbook Chapter 5 on image generation for full detail

Google and Microsoft RESTful Services

Example in Text Section 7.3.6

- Google map services
  - https://developers.google.com/maps/documentation/webservices/
- Microsoft Bing map services
  - http://msdn.microsoft.com/en-us/library/ff701713.aspx

Google Maps APIs Web Services

The Google Maps web services are a collection of HTTP interfaces
maps applications. This guide serves only to introduce the web serv
different services. Individual documentation for each service is loca

- Google Maps Directions API
- Google Maps Distance Matrix API
- Google Maps Elevation API
- Google Maps Geocoding API
- Google Maps Geolocation API
- Google Maps Roads API
- Google Maps Time Zone API
- Google Places API Web Service

Bing Maps REST Services

Getting Started with the Bing Maps REST Services	Gets you started with the Bing Ma
What's New in the REST Services	Describes the latest updates to th
Getting Traffic Incident Data	Shows how to get traffic incident
Using the REST Services with .NET	Shows how to use .NET with the R
Locations API	Shows how to geocode and rever
Elevations API	Shows how to get elevation inforr
Imagery API	Shows how to use the Imagery AP such as map tile URLs and imager

Google Map Service Call Example

- http://maps.googleapis.com/maps/api/distancematrix/xml?origins=Phoenix+AZ|Tucson+AZ&destinations=Los+Angeles|San+Francisco%&mode=driving&language=en&sensor=false

Add your API key to authenticate each request

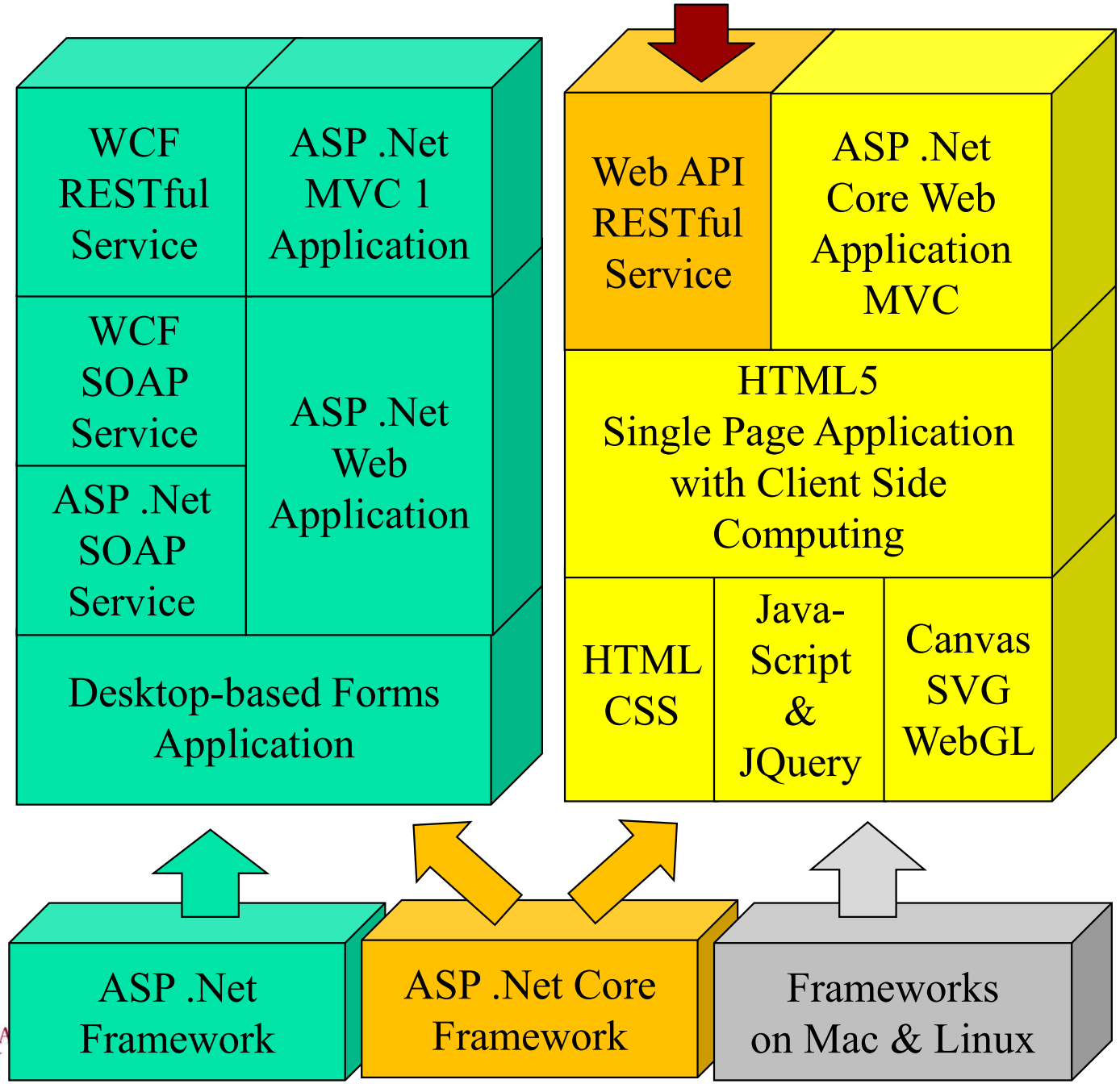

```xml
<?xml version="1.0" encoding="UTF-8"?>
- <DistanceMatrixResponse>
    <status>OK</status>
    <origin_address>Phoenix, AZ, USA</origin_address>
    <origin_address>Tucson, AZ, USA</origin_address>
    <destination_address>Los Angeles, CA, USA</destination_address>
    <destination_address>San Francisco, CA, USA</destination_address>
  - <row>
    - <element>
        <status>OK</status>
      - <duration>
          <value>19217</value>
          <text>5 hours 20 mins</text>
        </duration>
      - <distance>
          <value>598638</value>
          <text>599 km</text>
        </distance>
      </element>
    - <element>
        <status>OK</status>
      - <duration>
          <value>38771</value>
          <text>10 hours 46 mins</text>
        </duration>
      - <distance>
          <value>1211572</value>
          <text>1,212 km</text>
        </distance>
```


Distance / Time	Los Angeles	San Francisco
Phoenix		
Tucson		

Lecture Roadmap

- **Developing REST Services:**
  - Developing RESTful Service
  - Defining Input and Output Formats
- **Image Verifier in RESTful Service**
  - RESTful Service of a Random String Generator
  - RESTful Service of an Image Verifier
  - TryIt page calling RESTful Services
- **RESTful services and Applications in ASP .Net Core**
  - Creating cross platforms services and applications

ASP .Net Core Framework



Using ASP .Net Core Web Application

- New → Project …

New Project

Recent

Installed

Visual C#

Windows Universal

Windows Classic Desktop

Web

.NET Core

.NET Standard

Cloud

Test

WCF

Not finding what you are looking for?

Open Visual Studio Installer

.NET Framework 4.6.1 | Sort by: Default | Search (Ctrl+E)

ASP.NET Core Web Applicati... Visual C#

ASP.NET Web Application (.... Visual C#

Type: Visual C#

Project templates for creating ASP.NET Core applications for Windows, Linux and macOS using .NET Core or .NET Framework.

Name: WebApi

Location: Choose your directory for storing your project

Browse...

Solution name: WebApi

☑ Create directory for solution

☐ Create new Git repository

OK | Cancel

Choose Web API Template

New ASP.NET Core Web Application - WebApi

.NET Core | ASP.NET Core 2.0 | Learn more

Empty | Web API | Web Application | Web Application (Model-View-Controller) | Angular

React.js | React.js and Redux

A project template for creating an ASP.NET Core application with an example Controller for a RESTful HTTP service. This template can also be used for ASP.NET Core MVC Views and Controllers.

Learn more

Change Authentication

Authentication **No Authentication**

Enable Docker Support

OS: Windows

Requires Docker for Windows

Docker support can also be enabled later Learn more

OK | Cancel

Service Templates, Add Your Code …

WebApi | ValuesController.cs

WebApi | WebApi.Controllers.Valu | Put(int id, string value)

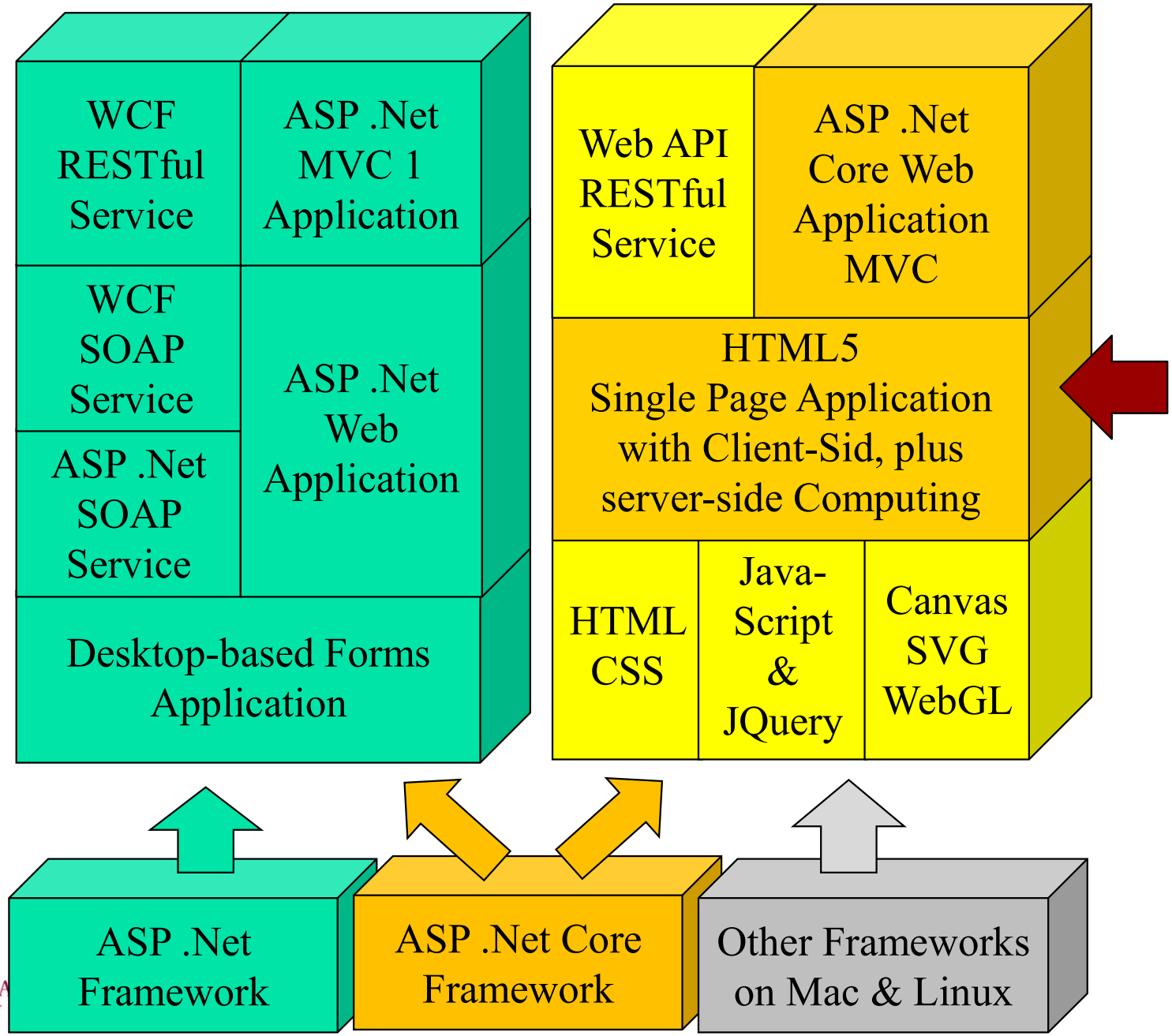
```
7   namespace WebApi.Controllers
8   {
9       [Route("api/[controller]")]
        0 references
10      public class ValuesController : Controller
11      {
12          // GET api/values
13          [HttpGet]
            0 references
14          public IEnumerable<string> Get()
15          {
16              return new string[] { "value1", "value2" };
17          }
18
19          // GET api/values/5
20          [HttpGet("{id}")]
            0 references
21          public string Get(int id)
22          {
23              return "value";
24          }
```

Solution Explorer

Search Solution Explorer (Ctrl+

- Solution 'WebApi' (1 project)
  - WebApi
    - Connected Services
    - Dependencies
    - Properties
    - wwwroot
    - Controllers
      - ValuesController.cs
        - ValuesController
    - appsettings.json
    - Program.cs
    - Startup.cs

Lecture Summary

- **Developing REST Services without using WCF:**
  - Developing RESTful API
  - Develop a gRPC Service and Client
  - Developing RESTful Service in Python
- **Case Study: Image Verifier in RESTful Service**
  - RESTful Service of a Random String Generator
  - RESTful Service of an Image Verifier
  - TryIt page calling RESTful Services
- **RESTful services in ASP .Net Core**
  - Creating cross platforms services and applications

Next Lecture